Slot-Chain: A Preconfirmation Protocol

Audited Design Candidate — v2.20

August 2026

Abstract

Slot-Chain assigns one-second L2 slots to bonded builders. Builders sign parent-linked blocks off chain; any party can prove and submit a batch to L1. Normal candidates compete for a fixed settlement interval. After an objective finality breach, or when a forced L1 message becomes due, anyone can prove an unsigned deterministic escape block. Recovery is an episode of renewable, objectively expiring rounds, so a prover outage does not make its target stale; as with every validity rollup, producing the next proof still requires a reconstructible canonical prestate. This revision separates proved outputs from L1 close metadata, replaces the unauthenticatable queue hash chain with a range-provable Merkle vector, makes forced-message execution Ethereum-valid, and specifies bounded admission, data, reward and migration state. It is an audited architecture candidate, not yet an implementation-ready specification or production authorization: the missing executable profile and the proving, gas, economics and adversarial gates in §13 remain mandatory.

Contents

1 Status, claims and exclusions

Normative words MUST, MUST NOTand SHOULDhave their usual RFC meaning. Where prose and either executable model disagree, deployment is blocked until the disagreement is resolved; neither source silently overrides the other.

1.1 Claims

Subject to L1 safety/liveness, a sound validity-proof system, availability of a canonical prestate reconstruction package as defined below, and—for kind-1 credits only—safety/finality of the supported source domain and its profile-authenticated, append-only Bridge registry, the protocol claims:

1. **Finalized-state safety.** L1 accepts only a proved state transition extending the stored canonical tuple. Builder signatures never substitute for execution validity.
2. **Bounded protocol recovery.** A builder cartel, an absent aggregator, an empty builder registry, and missing standbys cannot prevent an unsigned escape candidate from advancing the canonical state while the canonical prestate and any unexpired forced bytes remain reconstructible.
3. **Forced transaction inclusion.** A correctly prepaid L1 envelope at the queue head is consumed by the next canonical block after it becomes due; kind 0 is executed if valid bytes are supplied before its bounded validity or consumed-and-discarded otherwise, while durable kind 1 is always applied idempotently.
4. **Permissionless landing and recovery.** No allowlist, builder key, aggregator seat or payment recipient is required to submit a proof or an escape candidate.
5. **Deterministic lookahead.** Every implementation derives the same schedule from an authenticated finalized snapshot and the byte encoding in §4.

1.2 Non-claims

- A preconfirmation is not finality or insurance. Before normal close it may be revoked by builder equivocation, an unprovable skip, a withheld body, or an escape commit. Applications must price this as probabilistic/economic assurance and publish their own exposure limit.
- A bond cannot cap aggregate off-chain promises: the protocol has no reservation ledger and a builder can sign mutually exclusive promises. `L_LEASE` is deterrence and a recovery source, not guaranteed coverage.
- Protocol liveness is a capability claim. Economic liveness additionally assumes at least one actor can fund proof generation and an L1 transaction. A payment failure never invalidates an otherwise valid canonical transition.
- Censorship by L1 for longer than `T_INCLUDE_MAX_SECONDS`, failure of L1 finality, or a broken proof system is outside the model.
- A supported source domain finality failure or unauthorized change to its profile-pinned Bridge registry is outside the kind-1 safety claim. It cannot authorize a kind-0 user envelope.
- A source message is not yet an admitted forced envelope. Any preparer must supply its full Message preimage and queue deposit to the same-L1 adapter; source emission alone is

not reported as forced-queue acceptance. If nobody enqueues before the record's bounded `enqueueBy`, permissionless source cancellation returns its ETH escrow and permanently forbids later enqueue. Official token vaults participate only after installing the bounded durable refund-capsule path below; a legacy callback-only contract cannot originate a post-cutover V2 message and remains on the V1 compatibility path.

- The immutable V2 custody kernel is not self-healing. A defect in its compiled Bridge, vault, inbox store, accumulator, history router or fixed Merkle verifier cannot be patched for an already-live credit without introducing an authority able to forge delivery/failure and take reserved value. This design chooses fail-closed immutability; production status therefore requires the compiled-code, formal-invariant and differential-test gates in §13, not merely this document.
- A reserve and valid claim cannot force an externally governed ERC20, ERC721 or ERC1155 contract to transfer while that token is paused, blacklists the recipient or changes semantics. The protocol keeps the capsule and reserve retryable; asset-contract liveness is an explicit external condition.
- A state root is a binding commitment, not an erasure code. If every copy of the canonical state trie, executed bodies, runtime-bytecode preimages and required witness data is destroyed after Ethereum's applicable data-retention horizon, a new prover cannot reconstruct the preimage from L1 roots. Arbitrary-duration recovery after total canonical-data loss is therefore not claimed.

2 Actors and trust model

Builder.
A registered address eligible for scheduled slots. Builders may equivocate, skip parents, withhold bodies, collude, or disappear.

Aggregator.
The current auction seat, expected to collect data, prove and submit normal candidates. It has no exclusive consensus authority.

Prover/lander.
Any address. It supplies a validity proof and names an optional reward beneficiary in the proof statement. Proof validity, not identity, authorizes transition.

Canonical archive.
Any untrusted, replaceable source of canonical L2 bodies, state-trie nodes and runtime-bytecode preimages sufficient to reconstruct the current prestate. Each bytecode object is addressed by its legacy-Keccak `codeHash`; this includes code reached through proxy, beacon or delegation indirection. Content is verified against the L1-canonical block/state roots, so correctness does not require trusting the source; liveness requires at least one complete copy to remain reachable.

Forced-message sender.
Any L1 account that submits a gas-bounded, prepaid L2 transaction envelope.

Bridge source domain.
For kind 1 only at launch, settlement Ethereum, identified by chain ID, genesis hash, permanent frozen Bridge facade, non-proxy credit registry, immutable terminal verifier and namespace. The adapter direct-reads its append-only credit record on the same L1. Registry governance is an external safety dependency; a current resolver lookup never substi-

tutes for the emission record.

Protocol governance.
Delayed governance selects each validity verifier, execution profile and Settlement implementation. It is already a safety root: an authorized malicious verifier can finalize an arbitrary state transition. V2 recovery adds no stronger writer. Each immutable Settlement records its own proof-bound canonical history internally; the protocol-lifetime router only registers exact version/code identities and reads those histories. It has no governor, canonical-write callback or caller-supplied-root setter.

Bridge message archive.
Any untrusted, replaceable source of the full Message bytes emitted on L1. Correctness follows from `msgHash`; successful destination execution needs one copy until processing. Loss before `enqueueBy` leads to source cancellation; loss after the L2 pin leads to permissionless destination expiry and failure. Source ETH and official-vault assets remain recoverable by credit ID from their bounded durable refund records, without reconstructing Message bytes.

L1.
Ethereum provides canonical ordering, timestamps, EIP-4788 beacon roots, EIP-2935 historical block hashes, blob commitments and the KZG point-evaluation precompile.

The safety threshold contains no honest-builder assumption. Honest builders improve normal throughput and preconfirmation reliability; the unsigned escape lane provides state and forced-queue progress when all of them collude. The L1-liveness assumption includes transaction inclusion and at least 64 canonical L1 blocks becoming available within `T_DEPTH_MAX=900s`; a longer L1 stall suspends, rather than violates, the L2 recovery clock. Archive operators have no protocol key, seat or exclusive API. Anyone may mirror or serve the same content-addressed data. This is an availability assumption, not an authority assumption; it limits long-horizon cold-start recovery but does not weaken proof soundness or permissionless submission.

3 Clock domains and canonical state

Before activation, Settlement is in `PREACTIVE`; candidate submission is disabled and slot arithmetic saturates. L2 slot zero is fixed by `GENESIS_TIMESTAMP`, with

`currentL2Slot=max(0,block.timestamp-GENESIS_TIMESTAMP).`

L2 slot and every deadline are timestamp seconds. L1 confirmation depth is counted in L1 block numbers. Beacon slot is used only in schedule authentication. Implementations MUST NOTsubstitute one clock for another. `settlementChainId` is the L1 chain executing Settlement and domains every L1 contract commitment; `l2ChainId` is the EVM chain ID enforced for L2 transactions and execution. They are independent immutable launch constants and are both explicit in every cross-domain proof statement and builder header.

The L1 contract stores a proved core and local L1 metadata:

```
CanonicalCore = (l2BlockNumber, tipHash, tipSlot, stateRoot, messageCursor,
                 winningDataCommitment, nextBaseFee, nextExcessBlobGas,
                 terminalRoot, terminalCount)
Canonical = (CanonicalCore core, uint64 canonicalizedAtBlock)
```

The proof binds `baseCanonicalHash`, the hash of both stored objects, but outputs only a new `CanonicalCore`. Settlement compares every base field with storage, verifies the proof, writes the explicit proved core, and only then sets `canonicalizedAtBlock=block.number`. A prover is therefore never asked to predict the L1 block in which its transaction will land. No code attempts to read or store `blockhash(block.number)`, which does not exist during that block.

`l2BlockNumber` is the canonical EVM height, distinct from the one-second slot because slot gaps are legal. A candidate with `blockCount=n` proves `endL2BlockNumber=base.l2BlockNumber+n`; every executed header uses the next consecutive EVM block number. While the deployed `ICheckpointStore` uses `uint48`, launch and every transition additionally require `endL2BlockNumber<=UINT48_MAX`; changing that interface and bound is a protocol-version upgrade. The canonical event always emits `(l2BlockNumber,tipHash,stateRoot,terminalRoot,terminalCount)`. The proof constrains the latter pair to the exact post-state slots of the protocol-lifetime `TerminalAccumulatorV2`; Settlement never accepts them as unverified auxiliary calldata. In the same internal storage transition that writes `Canonical`, Settlement increments a checked `uint64canonicalSequence` and writes the complete sequence-tagged canonical read tuple to its 256-cell history ring. A second canonical transition requires `block.number>lastCanonicalCommitBlock`; it cannot overwrite two cells in one L1 block. PREACTIVE and FROZEN instances cannot execute any canonical path. Recording this ring is an internal write, not an external router call, reward dependency or best-effort publication. `nextBaseFee` and `nextExcessBlobGas` are the exact fork-rule outputs needed to construct the next header without retrieving a prunable historical parent body/header. The proof derives them from the last executed header; Settlement stores them but does not calculate fork arithmetic.

A candidate has one prior-block anchor. Normal activation stores the hash of its earlier arm block using native `blockhash`; candidate submissions hash the supplied execution header to that stored value and use MPT proofs, including the historical forced-queue root/count. EIP-2935 is used later to authenticate that arm hash for equivocation evidence. A recovery round instead stores `(block.number-1,blockhash(block.number-1))`, directly stores the current queue root/count, and requires that anchor to reach depth `F_L1`. The EVM does not expose the parent timestamp. At candidate submission Settlement hashes and parses the supplied RLP parent header, requiring its number/hash to equal the frozen pair; its state root and timestamp become verifier-statement fields but are not separately frozen scalars. Current queue state is never claimed to be part of the parent's historical state root. An L1 reorg rolls the stored canonical state and anchor back together. Initial values and bridge cursors are migration inputs in §11.

Every state-mutating external entry point begins with `sync()`. If `sync()` changes mode or commits a mature normal candidate, the wrapper MUSTreturn `SYNCED` successfully before parsing or validating the caller's candidate. The caller resubmits against the new version. This rule is necessary because a later Solidity revert would otherwise roll back the sync transition in the same transaction.

4 Builder registry and exact lookahead

4.1 Registry lifecycle

A registration contains a nonzero address, a `uint192` bond not exceeding `BOND_MAX`, a monotonic `uint64registrationIndex`, an `effectiveL2Slot`, and separately reserved

L_LEASE[window] tranches. Address zero is permanently VACANT. Duplicate live addresses and reuse while any prior generation remains active or liable are forbidden. Once the old generation's last evidence/reorg deadline has passed, all tranches are terminal, its liability leaf is safely cleared and no evidence can still land, the address may register again with a fresh registrationIndex. No permanent address tombstone mapping exists. With currentWindow=floor(currentL2Slot/384), a reservation is accepted only for currentWindow<=W<=currentWindow+MAX_TRANCHE_AHEAD_WINDOWS, where the initial ahead bound is 16; an exit request permanently disables new reservations for that generation. Only a present *active* generation with no exit request and no tombstone may execute FREE->RESERVED; a replacement or exit atomically closes reservations before moving the old generation into liability. Liability records may only be slashed or released, never extended with a new reservation.

The active table has exactly 64 indexed cells. On accepted registration, effectiveL2Slot= currentL2Slot+384*ENTRY_DELAY_WINDOWS; the stored value, not a later recomputation from a window boundary, is authoritative. Thus a registration becomes effective only after ENTRY_ DELAY_WINDOWS=8. If the table is full it may replace only the smallest-bond effective live entry, breaking ties by greatest registration index, and only with a strictly greater bond. At most MAX_ GENERATION_MOVES_PER_WINDOW=4 replacements or effective exits are processed. Exit requests receive a monotonic sequence and mature at requestWindow+EXIT_DELAY_WINDOWS. Every replacement path first processes the oldest matured exits by (matureWindow,exitSequence) within the window's remaining movement budget; if any matured exit remains, replacement rejects. Anyone, including the exiting builder, may run this bounded maintenance. Thus transaction ordering cannot starve an eligible exit. A displaced generation is moved, with its tranche root and bond, into the fixed MAX_LIABILITY_GENERATIONS=1,072 liability ring; an occupied ring cell is never overwritten before its release predicate holds. Registration therefore rejects, rather than losing slashable history, when the replacement budget or ring is full.

The six-level registryRoot over the 64 active cells commits each cell's mutable tranche root and therefore changes on reservation transitions. It is used only by schedule sealing. A separate eleven-level admissionRoot over 2,048 fixed positions contains stable generation identity for the 64 active and 1,072 liability records plus canonical empty padding; its leaves do *not* contain tranche roots. Only generation admission, movement, tombstoning, slashing or release increments admissionVersion and updates admissionRoot. A FREE/RESERVED/LIABLE tranche transition updates the tranche and registry roots but cannot invalidate signed blocks or proofs by changing the admission pair. The admission root retains displaced scheduled keys. Normal activation and every recovery round freeze the pair (admissionVersion,admissionRoot). Each scheduled signer carries an inclusion proof for a non-tombstoned record effective at that L2 slot; a slash cannot retroactively invalidate an already frozen window. Outside an already-open normal window or live recovery round, a tombstone makes sealed future tickets at or after its effective slot return VACANT; inside that frozen candidate context the stored admission pair remains authoritative until close/commit/expiry.

Admission positions are canonical: active cell i is always position i for $0 \leq i < 64$; liability ring slot j is always position $64 + j$ for $0 \leq j < 1,072$. A generation move uses j= movementSequencemod1072. The transaction first proves/releases the old occupant, writes the moved generation at that same slot, clears its active cell or writes the replacement, increments movementSequence, then updates both affected roots and increments admissionVersion; the new admission pair is published atomically. Reservation-only mutations publish only the registry root and leave that pair byte-for-byte unchanged. There is no first-free search, compaction

or caller-chosen liability position.

Eligibility is evaluated before ranking. A cell's effective L2 slot must be no later than the source-derived L2 slot, its fully reserved tranche for W must be proven, and its tombstone-effective L2 slot must be later than the source. All 64 registry cells are supplied; only present cells supply tranche-ring proofs at `Wmod512`. A canonical `EMPTY` tranche leaf is valid input and makes the cell ineligible; it need not claim window W. A nonempty leaf with a different stored window is likewise ineligible, not a seal revert. Only an exact `RESERVED` leaf with stored window W and the full `L_LEASE` amount is eligible. The 64 eligible entries with greatest bond, then smallest registration index, form the snapshot set. Supplying proofs only for purported winners is invalid because it cannot prove omission-free ranking.

An exit request becomes eligible for processing only after `EXIT_DELAY_WINDOWS=MAX_LIVE_WINDOWS`; it may wait behind the same bounded FIFO of older matured exits, and the bond remains escrowed until every generation and tranche is releasable. A tranche follows `FREE->RESERVED(W)->LIABLE(W)->RELEASED` or `RESERVED/LIABLE->SLASHED`. Every candidate block slot must be at least the normal window's frozen minimum (or the recovery round start), not merely its tip. Let `windowEndSlot(W)=384*(W+1)-1`. Release requires `windowEndSlot(W)<globalMinReferencedSlot`, no stored best or active round references it, and the absolute evidence and L1-reorg deadlines have expired. At that point maintenance atomically marks the schedule cell `EXPIRED` and may release its tranches; an unexpired sealed record is never overwritten. `evidenceDeadline(W)=GENESIS_TIMESTAMP+384*(W+1)+EVIDENCE_DELAY_SECONDS`; `evidenceReplayDeadline(W)=evidenceDeadline(W)+REORG_MARGIN_SECONDS` is both the contract's last admissible evidence time and the tranche's absolute release boundary. Evidence intended to survive a reorg must first be submitted by `evidenceDeadline(W)`. `globalMinReferencedSlot` is the minimum of the current dynamic floor, an open normal window's frozen floor, an active round's start and the first slot of any stored best (absent values are ignored). Slot indices are never compared with timestamps or window numbers.

The liability-capacity proof is a launch invariant, not an assertion by inspection. A generation moved in window C can have no reservation beyond `C+MAX_TRANCHE_AHEAD_WINDOWS`. Therefore

```
MAX_LIABILITY_RESIDENCE_WINDOWS =
  MAX_TRANCHE_AHEAD_WINDOWS + 1 +
  ceil((EVIDENCE_DELAY_SECONDS + REORG_MARGIN_SECONDS) / 384) + 2
```

must be strictly less than `MAX_LIVE_WINDOWS=268`. The two extra windows cover boundary ordering and bounded maintenance. At four moves per window, the ring position reused 268 windows later is consequently releasable. Each generation maintains an exact unreleased-tranche counter, so release and reuse do not scan 512 leaves.

4.2 Authenticated snapshot

For aligned window W, let its start timestamp be `GENESIS_TIMESTAMP+384W`. The target is the greatest `BEACON_GENESIS_TIME+12k` not later than eight L1 epochs before that start; L2-genesis alignment is irrelevant. Define `sealDeadline(W)=windowStart-H_LOOK`. A seal is valid only when its carrier execution block has `F_FINAL` L1-block depth and `block.timestamp<sealDeadline(W)`. Beacon slots are not used as a surrogate for execution-block depth. At or after the deadline an unsealed window is permanently all-`VACANT`; a late

transaction cannot change it. The contract performs this procedure:

1. The contract itself, not caller data, queries EIP-4788 for $j = 1, \ldots, 64$ at `q=targetTimestamp+12j`; missing timestamps revert and are skipped. The first successful query identifies the *carrier* execution block. EIP-4788 returns its parent beacon-block root, not the carrier root.
2. For the first successful query the caller supplies the carrier execution header. EIP-2935 authenticates its hash and block number; the header timestamp equals q and its `parent_beacon_block_root` equals the EIP-4788 result. Fork-specific SSZ branches in that parent authenticate its slot and one execution payload's `blockHash`, `stateRoot`, `prevRandao` and timestamp. The parent slot must be at or before target, its payload timestamp must equal its beacon-slot time, and its payload block hash must equal the carrier header's parent hash. No carrier observed by all 64 contract calls means all `VACANT`. If any call succeeds, a malformed header, a later parent, or a fork/gindex mismatch reverts without recording state; adversarial witness bytes can never manufacture a vacant seal.
3. Verify Merkle–Patricia storage proofs from that execution state root for the registry header and `registryRoot`. The caller supplies all 64 canonical serialized cells, including empty cells; recompute their six-level Keccak tree and require equality to `registryRoot`. Filter eligibility and order the entries, then store the byte-exact 64-leaf `entryRoot` and seed. Every *present* active cell supplies its tranche leaf and nine-sibling proof at index `Wmod512`. An absent registry cell has `trancheRoot=0`, supplies no tranche proof and is synthesized ineligible. For a present cell, canonical `EMPTY`, `FREE`, or a nonempty leaf for a different stored window is valid but ineligible; only exact `RESERVED(W)` with `amount=L_LEASE` is eligible. Proofs occur in ascending cell-index order, and extra/missing proofs reject. The fixed cell count proves completeness with one MPT path rather than 64 independent storage paths. The carrier satisfies `currentExecutionBlock-carrierBlockNumber>=F_FINAL`; the source payload is necessarily older.

`sealWindow` is permissionless. An optional maintenance distributor may pay its event in a later transaction. A missing seal fails closed to `VACANT`; it cannot give an address an unauthorized ticket. The snapshot age plus 64-slot carrier scan, finality and seal margin fits the eight-epoch geometry and remains far below EIP-4788's 8,191-slot history window. Consensus constants include the Deneb SSZ generalized indices; an L1 fork that changes them requires a protocol-version upgrade, never runtime guessing.

For Deneb, Electra and Fulu, the generalized index from `BeaconBlock` root to `slot` is 8 and to `body.execution_payload` is 201. Within the composed tree, payload `state_root`, `prev_randao`, `timestamp` and `block_hash` are 6,434, 6,437, 6,441 and 6,444 respectively. The seal input names the L1 fork digest; only these configured digests are accepted. Gloas or any fork that changes the container requires new constants, fixtures and a protocol-version upgrade before its activation.

4.3 Consensus byte encoding and allocation

All hashes use Ethereum legacy Keccak-256. Concatenation below is packed fixed width; strings are the literal ASCII bytes without a length prefix:

$$
\begin{aligned}
seed_W &= H(\texttt{"slot-chain-seed-v1"} \| u256(settlementChainId) \\
&\quad \| u256(protocolVersion) \| u256(W) \| bytes32(prevRandao)), \\
qTie_i &= H(\texttt{"slot-chain-quota-tie-v1"} \| seed_W \| u64(regIndex_i)), \\
sTie_{p,i} &= H(\texttt{"slot-chain-slot-order-v1"} \| seed_W \| u16(p) \| address20(i)).
\end{aligned}
$$

Starting from zero, capped D'Hondt awards each of 384 tickets to the unsaturated entry maximizing `bond/(allocation+1)`, with `Q_MAX=76`. Quotients are compared by cross multiplication; a `uint192` bond times `Q_MAX+1` cannot overflow `uint256`. Equal quotients use smallest `qTie`. Remaining tickets are `VACANT`; six addresses are required to fill a window.

Placement processes positions in order. A real address is feasible if it has a ticket remaining, differs from the preceding real address, and removing it preserves `remaining[a] <=otherRemaining+1` for every real address. Choose the feasible value with smallest `sTie`; `VACANT` may repeat. The executable model and its golden vectors are normative test fixtures.

5 Blocks, promises and proof statement

5.1 Signed header

A builder signs exactly:

```
(settlementChainId, l2ChainId, protocolVersion, verifyingContract,
 slot, parentHash,
blockHash, stateRoot, bodyRoot, anchorNumber, anchorHash,
forceRoot, forceCutoff, messageStart, messageEnd, dataManifestRoot,
coinbase, tier, contextId, admissionVersion, admissionRoot,
 episode, recoveryRevision, recoveryId)
```

The normative EIP-712 type string is:

```
SlotChainBlock(
 uint256 settlementChainId,uint256 l2ChainId,uint256 protocolVersion,
 address verifyingContract,
 uint64 slot,bytes32 parentHash,bytes32 blockHash,bytes32 stateRoot,
 bytes32 bodyRoot,uint64 anchorNumber,bytes32 anchorHash,bytes32 forceRoot,
 uint64 forceCutoff,uint64 messageStart,uint64 messageEnd,
 bytes32 dataManifestRoot,address coinbase,uint8 tier,
 bytes32 contextId,uint64 admissionVersion,bytes32 admissionRoot,
 uint64 episode,
 uint64 recoveryRevision,
 bytes32 recoveryId)
```

The hashed ASCII has one space between each field type and field name and no other whitespace; displayed line breaks, indentation and spaces after commas are typography only. The exact single-line literal and a standalone Keccak implementation are in `commitment-model.py`. `TYPEHASH` is legacy Keccak-256 of those ASCII bytes and equals the concatenation of these two fragments:

0xee6a8c8e31e8245cd527869508f6e464
d60848939912038 76f734d1855aed87c

The EIP-712 domain is `("SlotChain","2",settlementChainId,verifyingContract)`. The explicit `l2ChainId` is the EVM `CHAINID` used by raw L2 transactions. Signatures are exactly 65-byte `r||s||v`; `v` is 27 or 28, s is low, and zero or recovered-zero addresses reject. EIP-2098 and EIP-155 transaction-style `v` values are not accepted. Tier 1 requires

```
contextId = H("slot-chain-normal-context-v1" || baseCanonicalHash ||
              u64(admissionVersion) || admissionRoot ||
              u64(armBlockNumber) || armBlockHash)
```

and its execution anchor is exactly that arm block. `episode=recoveryRevision=0` and `recoveryId=0`; tier 2 requires the exact live round values and `contextId=recoveryId`. The unsigned tier 3 uses that same recovery context. Consequently semantically unused bytes cannot create false equivocation evidence. The execution `blockHash` excludes the off-chain builder signature, avoiding a circular hash definition. The signed `bodyRoot` covers only canonical discretionary transaction bytes; the proof derives the full Ethereum transactions root after adding the profile-defined system and valid forced transactions.

5.2 Three validity tiers

For every tier and every block, the circuit requires the EVM header `timestamp=GENESIS_TIMESTAMP+slot` with checked addition, and derives `blockHash` from that exact header. The signed relative slot can therefore never disagree with EVM time, forced-expiry classification, or the canonical tip clock. Every signed block also carries the exact frozen `admissionRoot`; the circuit checks it together with `admissionVersion`.

1. **Normal signed.** Every block is signed by its scheduled builder under the normal window's frozen `(admissionVersion,admissionRoot)`. Slots strictly increase, each slot is no later than `currentL2Slot+CLOCK_SKEW`, and the first slot strictly exceeds the canonical base slot. Every block is at or above the window's frozen `minAdmissibleSlot`; parent gaps are at most `G_MAX=3,600`, equal to the objective final-lag trigger and within the 12-window schedule-proof cap.
2. **Recovery signed.** The first block extends the L1-final canonical tuple, binds the active episode and current recovery revision, and may cross an unbounded time gap. Every block still requires its scheduled builder signature under the current recovery round's frozen admission pair. Every slot is at or after that round's `roundStartSlot`.
3. **Escape unsigned.** Exactly one deterministic block extends the L1-final canonical tuple during recovery. It has no builder signature, no beneficiary-controlled coinbase, no discretionary transaction and no blob body. It contains only the protocol anchor and the deterministic maximum prefix of the current round's frozen forced queue. When the queue is empty an SLA-triggered episode may commit an empty anchor-only escape block. Its slot is `max(roundStartSlot+ESCAPE_OFFSET,canonical.tipSlot+1)` and its anchor, queue root/cutoff, block hash and state root are unique functions of the current recovery round and proved execution result.

Every candidate uses exactly one `batchAnchor`; every block repeats that anchor and frozen `forceRoot/forceCutoff`. For a normal candidate the supplied RLP execution header hashes

to the EIP-2935 value, its timestamp is no later than the first L2 block timestamp, and MPT proofs under its state root authenticate the historical forced queue and any other contract state. For recovery, the RLP header authenticates the stored parent number/hash and derives its timestamp and state root; the queue pair is authenticated directly by the round fields and is not asserted against that header's state root. Requiring one anchor, not up to 4,096 independent anchors, bounds both circuit and L1 work. Cutoffs therefore do not change within a candidate; the first blocks drain the frozen FIFO prefix and later blocks see an empty prefix.

The circuit verifies parent linkage, strict slot monotonicity, schedules, signatures for tiers 1–2, version binding, anchor/header/MPT authentication, forced-prefix ordering, execution, exact body/data coverage and public outputs. It rejects more than 4,096 blocks, 12 schedule windows, 16 sealed sessions, 2,100 referenced blob records, 256 forced items, 4 MiB forced bytes or 80 million total accounted forced gas per candidate. These candidate-level caps are independent of the per-block caps.

5.3 Verifier statement ABI

The L1 verifier accepts `verify(proof,uint256[2]digestLimbs)`. Settlement constructs the following Solidity struct in exactly this order and computes `statementHash` as legacy Keccak of the ASCII domain `"slot-chain-statement-v2"` followed by `abi.encode(S)`:

```
S = (uint256 settlementChainId, uint256 l2ChainId, uint256 protocolVersion,
     bytes32 executionProfileHash, address verifyingContract,
     uint8 tier, bytes32 baseCanonicalHash,
     bytes32 candidateCommitment, uint64 blockCount,
     uint64 firstSlot, bytes32 tipHash, uint64 tipSlot,
     uint64 endL2BlockNumber, bytes32 endStateRoot,
     bytes32 endTerminalRoot, uint64 endTerminalCount, uint64 endCursor,
     bytes32 winningDataCommitment, uint64 nextDueAt,
     uint256 nextBaseFee, uint64 nextExcessBlobGas,
     uint64 anchorNumber, bytes32 anchorHash,
     bytes32 anchorStateRoot, uint64 anchorTimestamp,
     bytes32 forceRoot, uint64 forceCutoff,
     bytes32 forcedDescriptorCommitment,
     uint64 startCursor,
     uint64 admissionVersion,
     bytes32 admissionRoot,
     uint64 episode, uint64 recoveryRevision, bytes32 recoveryId,
     bytes32 scheduleRootsCommitment, uint8 scheduleWindowCount,
     bytes32 sessionRefsCommitment, uint8 sessionCount,
     uint16 dataRecordCount, bytes32 executionOutputsCommitment,
     address proofBeneficiary)
```

`digestLimbs[0]` and `[1]` are respectively the high and low 128 bits of `statementHash`; neither is reduced modulo the proof field. The circuit recombines them and proves the exact preimage. Settlement independently checks the immutable `protocolVersion->executionProfileHash` mapping, the base canonical state, current/frozen versions and recovery round, recomputes the bounded schedule-root and sealed-session-reference commitments from calldata, recomputes the bounded forced-descriptor commitment from authoritative queue storage, recomputes `winningDataCommitment` from the candidate and sealed-session commit-

ments, requires `endL2BlockNumber=stored.l2BlockNumber+blockCount`, and constructs the new core from the explicit `(endL2BlockNumber,tipHash,tipSlot,endStateRoot, endTerminalRoot,endTerminalCount,endCursor)` fields plus that recomputed value and proved `(nextBaseFee,nextExcessBlobGas)`. It recomputes `executionOutputsCommitment` including the same explicit terminal pair; the circuit proves that pair equals the accumulator's exact post-state slots, so neither calldata nor Settlement can substitute a root. It rejects zero/ill-typed fields, cursor regression, or disagreement with the last proved block, then stamps `canonicalizedAtBlock=block.number` outside the proof. `candidateId=statementHash`. `nextDueAt` is authenticated inside the frozen queue as the boundary leaf's deadline, or `UINT64_MAX` when `endCursor=forceCutoff`. It is only a proof-consistency output. Normal submission additionally verifies the current-root boundary proof described in §7. After every commit and on every activation check, the sole authoritative live head value is `ForcedQueue.dueAt(canonical.messageCursor);` no recovery-round or canonical-core scalar caches it. `proofBeneficiary` is a proof input, never builder-signed block data or coinbase. Copying proof bytes cannot redirect a later optional reward.

5.4 Preconfirmation semantics

A builder signature proves authorship and makes same-slot equivocation slashable. It does not prove that the body reached a prover or that the next builder saw it. A later scheduled builder may sign a profitable skip that the protocol cannot distinguish from non-receipt. Accordingly:

- before L1 candidate acceptance, a promise is *observed*;
- after acceptance and before close, it is *provisionally proved*;
- after normal close or recovery commit, it is *canonical*; and
- only the underlying L1 finality policy makes it *L1-final*.

Wallets and applications MUST NOTdisplay the first two states as final. An escape commit explicitly revokes every omitted observed or provisionally proved promise and returns omitted transactions to the mempool.

6 Blob data authentication

Data is posted into publisher-owned, bonded sessions. A session ID is

```
H("slot-chain-session-v1" || u256(settlementChainId) || address20(contract) ||
  address20(owner) || u64(ownerNonce))
```

At most two sessions per owner and 1,024 globally may be live; each contains at most 2,100 records. Opening uses the first free ring cell after at most `MAX_GC_STEPS=8` deterministic expiry checks from a persistent cursor; callers may run bounded maintenance repeatedly. No call scans or clears all 1,024 cells. A full live ring rejects discretionary publication but cannot block the forced/escape lane. Dynamic rent, charged for the entire TTL and burned into local accounting, prices global occupancy. A Sybil willing to rent every cell can censor this optional DA market for one TTL; this is an explicit residual, not a protocol-liveness claim.

Only the owner may append, and a candidate may reference only a session made immutable by `sealSession`. Sealing fixes `(root,count,expiry);` there is no unseal or append-after-seal operation. A candidate supplies at most 16 sorted unique `(sessionId,count,root)`

references. At submission each must equal a live sealed session. Normal acceptance requires `expiry>=normalDeadline+REORG_MARGIN_SECONDS`; recovery requires `expiry>=block.timestamp+REORG_MARGIN_SECONDS`. Opening requires its expiry to cover proving, settlement and the reorg margin:

$$\begin{aligned} expiry &\geq now + P\_PROVE\_MAX + W\_SETTLE\_SECONDS \\ &\quad + REORG\_MARGIN\_SECONDS, \\ expiry &\leq now + DATA\_TTL. \end{aligned}$$

For every blob attached to `postData(session,manifests[])`, the contract:

1. obtains `versionedHash=BLOBHASH(i)` from the same transaction;
2. computes legacy Keccak over this packed fixed-width sequence:

   ```
   "slot-chain-data-fs-v2" || u256(settlementChainId) ||
   u256(protocolVersion) ||
   sessionId || versionedHash || fullBodyRoot || u16(blockOrdinal) ||
   u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
   chunkRoot || address20(publisher) || u64(validUntil)
   ```

   It interprets the digest big-endian and reduces it modulo the BLS scalar-field modulus to obtain z;
3. calls the EIP-4844 point-evaluation precompile with exactly `versionedHash32||z32||y32||commitment48||proof48`; and
4. appends the exact leaf from §A to the session MMR.

The proof receives blob polynomials privately, recomputes $P(z) = y$, and decodes the canonical blob codec. A discretionary body byte string is `u32(txCount)||(u32(txLength)||signedRawTx)*`; its root is `H("slot-chain-body-v1"||u32(byteLength)||bodyBytes)`. Blob payload is `u32(chunkByteLength)||chunkBytes||zeroPadding`, split into 31-byte chunks; each blob field element is `0x00||chunk31`. Exactly 4,096 elements are used, unused payload bytes are zero, and noncanonical encodings reject. A manifest entry binds `(blockOrdinal,chunkIndex,chunkCount,chunkByteLength,fullBodyRoot,chunkRoot,sessionId,recordIndex)`. For each block entries are in strict chunk-index order, cover exactly `0..chunkCount-1`, have `chunkCount<=MAX_CHUNKS_PER_BODY=9`, and concatenate to at most `MAX_BODY_BYTES=1,142,748`. The circuit recomputes every chunk root, the full discretionary body root and then the full Ethereum transactions root. The signed `dataManifestRoot` is *per block*. Its leaves contain only that block's entries, use local positions `0..m-1`, repeat that block's candidate-wide `blockOrdinal`, and are sorted by chunk index. Candidate calldata concatenates these independently rooted lists in increasing block ordinal; no global manifest tree exists. Duplicate, extra, overlapping or uncovered data is invalid. `dataManifestRoot` is the exact per-block tree in Appendix A, not a free-form publisher value.

Data publication has no consensus-coupled reimbursement. A separate service may pay it, but empty payment state cannot revert canonical settlement.

6.1 Canonical reconstruction availability

Blob sessions make an in-flight candidate provable; their TTL and garbage collection are not permanent state archival. On canonical commit, full nodes must retain the canonical raw

bodies, receipts, Ethereum trie nodes and every runtime-bytecode preimage needed to execute from `CanonicalCore.stateRoot`. Bytecode is mandatory because an Ethereum account leaf authenticates only `codeHash`, not the corresponding bytes. A reconstruction package is a content-addressed set of those objects plus the canonical block headers; each code object is keyed by `H(runtimeBytecode)` and must cover every account reachable during replay, including implementation code reached through proxy, beacon or delegation indirection. A consumer rejects any object whose transaction/body/header hash, trie path, bytecode hash or final state root disagrees with the L1-canonical commitments, so an archive can be anonymous and malicious without gaining safety authority.

Renewing a recovery round refreshes its L1 anchor, queue cutoff and proof target; it cannot invert a state root or recreate destroyed bytes. Therefore the 2,164-second protocol bound begins only once a prover has the canonical reconstruction package and any required unexpired kind-0 bytes. A warm full node satisfies this condition; a cold-start prover is bounded by data download plus proof time while at least one archive or Ethereum's applicable DA history still serves the package. If no complete copy exists, safety remains intact but progress stops. No governance action may replace the state root, skip the missing prestate, or call that condition a builder failure.

7 Settlement and recovery state machine

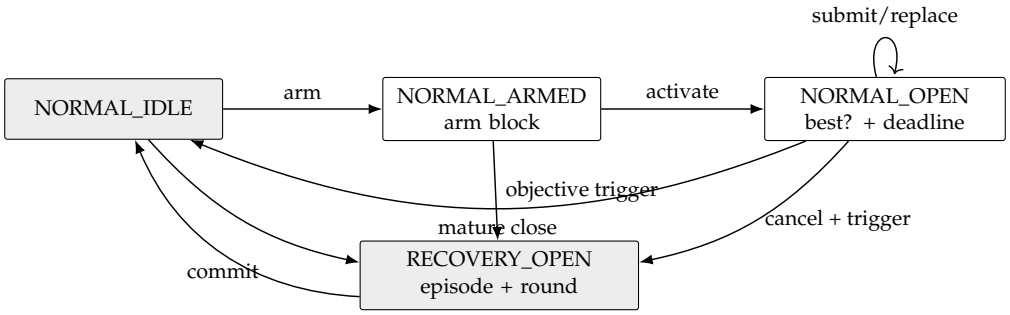


Figure 1: The complete mode machine. Payments are outside every arrow.

7.1 Normal context and candidates

Normal context creation is two-phase. Anyone calls `armNormalContext()` in `NORMAL_IDLE`; the call begins with `sync()`, stores only `armBlockNumber=block.number`, and emits `NormalContextArmed`. It accepts no caller-supplied anchor or hash. In a later L1 block anyone calls `activateNormalContext()`. It again begins with `sync()` and requires the arm's block-number age to be from 1 through `MAX_ARM_AGE_BLOCKS` (255), reads `armBlockHash=blockhash(armBlockNumber)`, and hashes a supplied RLP header to that value. It then freezes `baseCanonical`, `deadline=block.timestamp+W_SETTLE_SECONDS`, `minAdmissibleSlot=max(0,currentL2Slot-DELTA_TIP)`, the current stable `(admissionVersion,admissionRoot)`, the arm header fields, and `requiredThrough=deadline+T_INCLUDE_MAX_SECONDS`. It derives the sole normal `contextId` and emits `NormalContextActivated`. No candidate is accepted while armed. If age exceeds 255 blocks, any caller may invoke `armNormalContext()` again; after its leading `sync()` it atomically replaces only the stale arm number with the current block, emits `NormalContextRearmed`, and returns `REARMED`. A nonstale arm cannot be replaced. Thus an abandoned arm cannot suppress normal operation.

Builders collect and issue tier-1 signatures only after the activation event. The execution anchor for every candidate is the exact arm block, whose hash and parsed header are already frozen; replacements hash their supplied RLP header to that stored hash and never repeat an EIP-2935 recency lookup. A reorg that removes the arm block necessarily changes the context. If the arm block survives but the activation transaction is reorged, reactivation intentionally recreates the same context: the builder must retain and honor its prior same-context signature. Evidence independently authenticates the arm hash as canonical through EIP-2935 and is accepted only while that history entry is available. The enforced evidence horizon is strictly shorter than 8,191 L1 slots. Thus neither a shallow reorg nor a builder-selected historical anchor can create false or free equivocation.

Activation is permissionless, reserves no builder, and does not block submissions. An empty activated window merely cancels at deadline, so an opener gains no censorship right. Every candidate first proves its frozen-prefix boundary in the circuit. It also supplies a canonical single-leaf Merkle proof at `endCursor` under the queue's current root/count, or proves `endCursor==currentCount`. Settlement verifies at most 32 sibling hashes and requires the resulting `currentBoundaryDueAt` to exceed `requiredThrough`. This prevents an old but valid anchor from hiding a post-anchor append. Because due times are monotone, one current boundary leaf proves that every item due through the landing horizon is consumed. Later candidates use the same base, deadline, minimum, admission pair and horizon. Their total order is lexicographic:

$$(\text{block count descending},\ \text{tip slot descending},\ \text{tip hash ascending}).$$

Only a strictly better candidate replaces `normalBest`; the stored best contains `endCursor` and the minimum expiry of every referenced data session. Provisional replacement changes no canonical state and pays nobody. At or after the deadline, `sync()` reads `ForcedQueue.dueAt(best.endCursor)` and commits only if that live boundary is greater than `block.timestamp` and `block.timestamp+REORG_MARGIN_SECONDS<=best.minDataExpiry`. Thus a late close cancels rather than committing a block that omits an already-due head, even beyond the assumed inclusion bound, or whose authenticated data has passed its resubmission margin. Data-session GC cannot evict a session referenced by the stored best; its own leading `sync()` clears an expired best first. Because `FORCE_DELAY>=W_SETTLE_SECONDS+T_INCLUDE_MAX_SECONDS`, an append after open cannot violate the frozen acceptance horizon. Settlement then recomputes recovery causes against the new core.

7.2 Activation

While normal, `sync()` opens one persistent recovery episode if either:

- `currentL2Slot>canonical.tipSlot+DELTA_FINAL_LAG`; or
- the forced queue's stored `dueAt[canonical.messageCursor]` is no greater than `block.timestamp`.

A due forced head has precedence over an immature normal window, which is canceled. At a mature deadline, a best that consumes the required due prefix commits before causes are recomputed; an omitting best is canceled. No best is promoted across activation. The contract increments `episode`, records causes and the base-canonical hash, performs the SLA seat termination/burn at most once, then opens recovery round one. Termination and burn are

fixed-size writes in Settlement-owned escrow accounting: no token transfer, registry callback or caller-controlled external call occurs on this path. A later sweep may dispose of recorded burns.

Every round freezes `roundStartSlot`, the preceding L1 block number/hash, current `forceRoot/forceCutoff`, current `(admissionVersion,admissionRoot)`, and `escapeSlot= max(roundStartSlot+ESCAPE_OFFSET,canonical.tipSlot+1)`. Its `expiresAt` is `GENESIS_ TIMESTAMP+escapeSlot+DELTA_TIP`. The exact ID is:

```
recoveryId = H("slot-chain-recovery-v2",
               u256(settlementChainId), address20(contract),
               u64(episode), u64(revision), bytes32(baseCanonicalHash),
               u64(roundStartSlot), u64(anchorNumber), bytes32(anchorHash),
               bytes32(forceRoot), u64(forceCutoff), u64(admissionVersion),
               bytes32(admissionRoot), u64(escapeSlot), u8(causes))
```

At `block.timestamp>expiresAt`, and never before, `sync()` replaces the expired target with revision $r + 1$ and fresh round fields, then returns `SYNCED`. It does not change episode/base/cause, re-burn, terminate, promote or pay. The old proof is already inadmissible, so permissionless refresh cannot grief a still-live proof. New queue appends enter the refreshed cutoff. An envelope admitted during recovery receives `dueAt=max(enqueuedAt+ FORCE_DELAY,lastDueAt,currentRound.expiresAt+1);` therefore it cannot become due while a frozen round that omits it remains admissible.

7.3 Recovery acceptance

Recovery is first-valid, not heaviest. A tier-2 or tier-3 candidate MUST:

1. extend the exact current stored canonical state;
2. bind current `episode`, `revision`, `recoveryId`, frozen admission pair, anchor and queue target;
3. land at least `F_L1` L1 blocks after its anchor, have first slot at least `roundStartSlot`, tip no older than `DELTA_TIP`, and no slot later than wall clock plus skew;
4. consume the exact maximum forced-message prefix from the current cursor, bounded by the current round's frozen cutoff; and
5. satisfy its tier-specific signature/data restrictions and validity proof.

The first valid L1 transaction atomically commits the tuple, records the current L1 block *number only*, clears the recovery episode and returns to normal. Later proofs are stale. Candidate rejection cannot undo activation because of the early-return wrapper in §3.

8 Forced queue and unsigned escape

`ForcedEnvelope` is a tagged union. Kind 0 contains `(sender,nonce,l2ChainId,rawTxHash, byteLength,gasLimit,maxFee,validUntil,refundAddress,deposit)`. Admission canonically decodes the EIP-2718 transaction, requires its recovered sender and all duplicated fields to match, and requires `msg.sender==sender`; relayed admission needs a separately versioned EIP-712 authorization. Kind 1 contains:

```
(msgHash, srcChainId, sourceDomainId, srcEpoch, srcBridge,
 bridgeExecutionHash, emittedAtBlock, destinationDomainId,
 destChainId, enqueueBy, sender, srcOwner, destOwner,
 value, fee, calldataHash, refundMode, refundVault, refundCapsuleHash,
 escrowId, byteLength, accountedGas,
 refundAddress, deposit)
```

Only the configured `BridgeInboxAdapter` may enqueue it and it has no queue expiry. The permanent source Bridge endpoint retains the message value and fee while recording both `escrowLiability[creditId]` and aggregate `totalLiveLiability`. Every V1/V2 ETH outflow and governance sweep must leave `address(this).balance>=totalLiveLiability`; a bookkeeping entry without this solvency check is invalid. The destination adapter separately escrows only its caller-funded queue deposit. Its only forced effect is an idempotent inbox/signal credit keyed by the immutable identifier

```
sourceDomainId = H("slot-chain-source-domain-v4" || u64(srcChainId) ||
  bytes32(sourceGenesisHash) || address20(bridgeCreditRegistry) ||
  address20(terminalSignalVerifier) || address20(srcBridge) ||
  bridgeExecutionHash || bytes32(registryNamespace))
destinationDomainId = H("slot-chain-destination-domain-v6" ||
  u64(destChainId) || bytes32(destinationGenesisHash) ||
  address20(bridgeInboxAdapter) || address20(activeSettlementRouter) ||
  address20(terminalSignalVerifier) || address20(inboxApplyRouterV2) ||
  address20(inboxCreditStoreV2) || address20(protocolReleaseAuthorityV2) ||
  address20(terminalDomainRegistrarV2) ||
  address20(terminalAccumulatorV2) || address20(destinationBridge) ||
  bytes32(destinationBridgeExecutionHash) ||
  bytes32(destinationInfrastructureHash) ||
  bytes32(destinationNamespace))
creditId = H("slot-chain-bridge-credit-id-v5" || u64(srcChainId) ||
  sourceDomainId || u64(srcEpoch) || address20(srcBridge) ||
  destinationDomainId || msgHash)
escrowId = H("slot-chain-bridge-escrow-v1" || creditId)
```

Both domain descriptors are append-only and source-approved; no message sender supplies an arbitrary destination trust root. The destination descriptor binds the full stable recovery path: L2 genesis, non-proxy `BridgeInboxAdapter`, immutable `ActiveSettlementRouter`, `TerminalSignalVerifier`, `InboxApplyRouterV2`, `InboxCreditStoreV2`, protocol-lifetime `ProtocolReleaseAuthorityV2`, immutable `TerminalDomainRegistrarV2`, protocol-lifetime `TerminalAccumulatorV2` and frozen destination Bridge facade. The destination infrastructure hash is the Appendix-defined commitment to the deployed runtimes and constructor immutables of every one of those components; address-only binding is insufficient. The source and

destination Bridge addresses and official refund vaults are lifetime constants for that domain; neither resolver rotation nor a replacement proxy may change their callback, custody, status or terminal identity. An incompatible implementation requires a separately named endpoint and new domain, while every old endpoint remains callable for its old credits. Arbitrary destination invocation and RETRIABLE processing remain later ordinary bridge operations. The two forced-envelope variants have separate byte-exact leaves.

Launch accepts kind 1 only for the L1-to-slot-chain direction, where:

```
srcChainId = settlementChainId = block.chainid
```

Consequently the adapter omits an EIP-2935/MPT proof of its own L1. It performs a bounded `staticcall` directly to the descriptor's non-proxy `BridgeCreditRegistry`, checks its runtime hash and exact fixed-size return, and compares the immutable `CreditRecord`. A reorg that removes the source record necessarily removes every later enqueue transaction; anyone may retry from the still-durable record on the surviving chain. Cross-L1 sources require a separately reviewed protocol version and are not silently routed through this path.

The kind-0 v2 and kind-1 v10 leaf, descriptor and disposition decoders are protocol-lifetime queue ABIs. Every future execution profile must retain their exact semantics for all entries below `queueCount`; a migration may add a new tagged kind but cannot reinterpret an existing byte. The permanent queue keeps every fixed-width descriptor and kind-1 credit record. For kind 0, a candidate supplies raw EIP-2718 bytes matching `rawTxHash` while unexpired; if no party retains those bytes, the head becomes permissionlessly `EXPIRED_NO_TX` after the bounded seven-day validity horizon, using only the stored descriptor. Thus migration neither assumes an expiring blob for a durable bridge credit nor turns missing user bytes into permanent head-of-line blocking.

The deployed `sendMessage(Message)` selector, return layout, message-ID counter, `MessageSent` event, source signal and every V1 status/recall semantic remain byte-for-byte V1 for every caller after cutover. It never probes ERC-165 and never silently creates a V2 record. The additive `sendMessageV2(Message)` and `sendMessageFromVaultV2(Message)` selectors share the same checked monotonic message-ID counter, derives the delayed source-approved destination domain, locks `value+fee`, atomically asks `BridgeCreditRegistry` to create the V2 record, and emit only the profile-exact `MessageSentV2`; their return is the same `(msgHash,Message)` shape for tooling convenience, not ABI aliasing. The record sets `enqueueBy=block.timestamp+MAX_BRIDGE_ENQUEUE_DELAY`, initially 604,800 seconds, with checked `uint64` arithmetic. Before activation both V2 selectors revert. L2-to-L1 sending remains V1; the V2 forced-inbox switch is deliberately directional. A direct sender or upgraded application opts in with `sendMessageV2`, which always records `refundMode=DIRECT`, `refundVault=address(0)` and status `NEW`. Official vaults expose separate `sendTokenV2` entry points and call only `sendMessageFromVaultV2`, which always records `refundMode=CAPSULE`, `refundVault=msg.sender` and status `DRAFT`. They choose V1/V2 per asset; an old token without refund restoration can therefore remain V1 even though the vault also supports V2. A V2 vault entry point reverts unless its same-transaction context query returns the newly created credit before capsule finalization. A direct caller cannot request CAPSULE mode and a vault cannot nominate another refund vault. No code-size, interface, resolver or allowlist heuristic selects the committed mode.

`BridgeCreditRegistry` recomputes `creditId`, requires its immutable frozen-Bridge caller, loads the one final source generation and execution hash, and rejects duplicate IDs. The same transaction records the Bridge's per-credit and aggregate liabilities; `escrowId` is the committed

external identifier, not a second storage key. The vault-only selector first creates a DRAFT; after sendMessageFromVaultV2 returns in the same outer transaction it persists a bounded capsule keyed by creditId and calls finalizeRefundCapsuleV2. The Bridge checks the caller and capsule hash and changes DRAFT->NEW. Official vault entry points revert the entire outer transaction unless this succeeds. Direct-refund messages are born NEW. Only NEW records authorize enqueue. Storage is deliberately split. The non-proxy registry owns this immutable CreditAuthorization:

```
(bytes32 sourceDomainId, uint64 srcEpoch, address srcBridge,
 bytes32 bridgeExecutionHash, uint64 emittedAtBlock, bytes32 msgHash,
 bytes32 destinationDomainId, uint64 destChainId,
 uint64 enqueueBy, address sender, address srcOwner, address destOwner,
 uint256 value, uint64 fee, bytes32 calldataHash, uint8 refundMode,
 address refundVault,
 bytes32 escrowId)
```

The permanent Bridge owns mutable state:

```
SourceLiability(value, fee, refundCapsuleHash,
                status, queueIndex, pullAmount)
```

The authorization is immutable immediately; only the liability may move DRAFT->NEW->QUEUED->DELIVERED/RECALLED or DRAFT/NEW->CANCELLED. The profile pins both mapping slots and every packed member offset. A bounded getEnqueueRecordV2(creditId) staticcall returns their exact joined fixed-size view; the adapter cross-checks every duplicate field, exact NEW status and aggregate solvency. After append it calls the source Bridge from the adapter bound in destinationDomainId; markQueuedV2 atomically records the exact queue index and changes NEW->QUEUED. Any later revert unwinds the queue append and source transition together. Cancellation reads only this permanent source status—never replaceable adapter-local state—and changes DRAFT/NEW to CANCELLED after enqueueBy, crediting value plus fee to the source owner's pull balance. At QUEUED the fee reservation becomes ordinary bridge liquidity, while value remains reserved. The destination appends a permanent terminal DONE or FAILED accumulator leaf. Permissionless finalizeDoneV2 proves DONE and changes QUEUED to DELIVERED, releasing the value reservation; permissionless recallMessageV2 proves FAILED, changes QUEUED to RECALLED and credits value to the source owner's pull balance. Terminal state is written before crediting, and withdrawal is a separate checks-effects-interactions call.

The L1 BridgeDomainRegistry retains append-only source-domain, destination-domain and frozen-endpoint-support descriptors. Only the delayed ProtocolVersionManager may stage a descriptor, atomically with the active immutable release manifest. A staged tuple is unusable until permissionless confirmDestinationRegistration(protocolVersion, canonicalSequence,proof) atomically reads the exact sequence-tagged full canonical tuple through ActiveSettlementRouter, requires its L1 canonicalization block to be at least F_L1 deep, and authenticates the exact domain-registration record under that tuple's stateRoot. It never accepts a root or tuple from the caller. proof is MptProofV1: big-endian u16(accountNodeCount)||u16(storageNodeCount) followed by each root-to-leaf canonical-RLP node as u16(nodeLength)||nodeBytes. Each path has at most 66 nodes, each node at most 600 bytes, the whole proof at most 80,000 bytes, and any noncanonical RLP, wrong inline/hash reference, surplus node/byte or gas use above

8,000,000 rejects. The account proof fixes the registrar address and code hash; the storage proof fixes one nonzero `registrationCommitment[protocolVersion]` word. No Solidity struct or adjacent slot is inferred from that one path. The registrar exposes `registrationCommitmentV2(uint64)viewreturns(bytes32)` with exact 32-byte returndata, and stores only:

```
registrationCommitment =
  H("slot-chain-destination-registration-v1" ||
    u64(protocolVersion) || releaseManifestHash ||
    u256(destinationChainId) || manifestNamespace ||
    destinationDomainId || address20(destinationBridge) ||
    destinationInfrastructureHash || executionProfileHash)

baseSlot = H("slot-chain-terminal-domain-registrar.registrationCommitment.v1")
elementSlot = H(bytes32(uint256(protocolVersion)) || baseSlot)
storageTrieKey = H(elementSlot)
```

The storage-trie leaf value is canonical RLP of the minimal big-endian unsigned integer represented by the nonzero 32-byte commitment (and therefore preserves leading-zero semantics through the expected value). L1 recomputes the full commitment from its staged release manifest and accepts only exact equality; substitution of any field, mapping key, code hash, account or root rejects. The confirmation block, not manifest staging, starts the 214-block support delay. Each release adds at most 64 entries; there is no global cap, deletion, redirect or reinterpretation. A conflicting duplicate or inactive manifest rejects. Every credit creation performs an O(1) check that its exact `(sourceDomainId,bridgeExecutionHash,destinationDomainId)` support entry has been active for 214 blocks; adding D2 never makes it usable with an old source endpoint before D2 itself is final.

At cutover the existing UUPS Bridge proxy is upgraded once to the final `BridgeV2Facade`; the proxy address remains the domain's Bridge forever. The facade contains the complete V1 and V2 state machine. It exposes no upgrade selector, generic storage primitive or caller-chosen call target and never `delegatecall`s another contract. The proxy's one unavoidable `delegatecall` therefore always targets the same profile-proved facade. The migration proof checks the EIP-1967 slot and the facade runtime; selector and fuzz corpora prove that no calldata can rewrite that slot. The official refund-vault proxies are frozen under the same rule. This prohibition is a consensus safety boundary: a scheduled delegatecall executor could overwrite the implementation, liability or capsule slots despite any dispatcher policy. A Bridge-shaped clone cannot create a record because the registry accepts only the exact frozen facade. The source generation is a fixed nonzero registration number, not an automatically activated implementation epoch. The source execution hash is

```
H("slot-chain-frozen-bridge-execution-v2" ||
  u32(156) || canonicalFrozenBridgeDescriptor)
```

where Appendix A fixes the bounded grammar. It commits the Bridge, credit registry, terminal verifier, facade runtime, storage layout and acyclic Bridge-kernel profile, not the full execution profile. The derivation order is kernel profile, source and destination descriptors/execution hashes, infrastructure hash, destination domain and finally the full execution profile; no field hashes a value that transitively contains itself. The registry binds that

generation and execution hash into each credit; no current-block dispatch or historical executor selection exists. At launch all of these contracts share Ethereum; cross-L1 support or a new Bridge endpoint requires a new reviewed profile and domain. The destination uses the separately domain-tagged 196-byte destination Bridge descriptor in Appendix A; no source-only descriptor or caller-injected hash may stand in for it. Admission also requires `0<srcChainId<=UINT64_MAX`, nonzero `sourceDomainId`, `0<srcEpoch<=UINT64_MAX`, `0<emittedAtBlock<=UINT64_MAX`, nonzero `msgHash`, `destChainId=l2ChainId<=UINT64_MAX`, a registered nonzero `destinationDomainId` and a nonzero `srcBridge`; no narrowing cast or destination-chain inference is permitted.

`BridgeInboxAdapter` is a non-proxy immutable contract. Its profile-bound runtime and constructor values fix the credit registry, source Bridge, immutable settlement router and ProtocolVersionManager. Its sole mutable configuration is a zero-to-nonzero `destinationDomainId` slot sealed once by that manager during manifest activation; the setter deletes its authority bit and cannot run twice. Enqueue rejects while the slot is zero. It has no upgrade, delegatecall, mutable target, arbitrary-call or caller-supplied authenticated value path. A different destination uses a distinct adapter/domain; an old adapter remains executable for its existing records.

The adapter is the exactly-once machine `NONE->QUEUED(index)`, keyed by `creditId`. Any caller supplies the separate queue deposit and the full message preimage. In one transaction the adapter derives both domains and the ID, direct-reads the immutable record and live source liability, validates all fixed bounds including `block.timestamp<=enqueueBy`, and calls `ActiveSettlementRouter`. If router `sync()` changes mode, that transition must persist: the adapter records no credit or queue leaf, credits the caller's entire `msg.value` to a pull-refund ledger and returns `SYNCED`. The caller retries in a later transaction. Otherwise the router checks `queueCount<UINT32_MAX`, appends at the current count, timestamps the deadline, the source Bridge changes NEW to QUEUED at that exact index, and the adapter records the returned index, all atomically. No reservation, source proof, PENDING state or finalization call exists. A repeated zero-value query returns the stored index; a duplicate with funds reverts. The source event alone never means queued.

Kind 0 requires a nonexpired `validUntil`, `validUntil<=enqueuedAt+MAX_FORCE_VALIDITY_SECONDS`, and `accountedGas=max(gasLimit,intrinsicGas,MIN_FORCE_ACCOUNTED_GAS)`. The initial validity horizon is seven days. Kind 1 uses the permanent conservative bound `accountedGas=MAX_FORCE_MESSAGE_GAS=5,000,000` for its encoded credit and routed pin. Kind 0 requires `0<accountedGas<=5,000,000`. Both kinds require `0<byteLength<=131,072` and a deposit covering execution, proof credit and permanent calldata/state costs. The adapter verifies kind-1 escrow and all static bounds before calling the router; failure leaves neither deposit, queue leaf nor partial credit.

Only `ActiveSettlementRouter` may append. It calls the active Settlement's `sync()` under a non-reentrant guard and reports `SYNCED` without appending if any transition occurred. Its adapter caller preserves that transition, creates no admission record and makes the entire supplied deposit immediately withdrawable by the original caller. Otherwise it computes `enqueuedAt=block.timestamp` and `dueAt=max(enqueuedAt+FORCE_DELAY,lastDueAt,recoveryExpiry+1)` (the last term is zero outside recovery), and appends atomically. `ForcedQueue` stores one authoritative `QueueDescriptor` per index. It is the complete fixed-width kind-0 or kind-1 leaf preimage in Appendix A, excluding only kind-0 `rawTx` bytes themselves but including their hash. Thus sender, nonce, chain IDs, byte/gas fields, fee, validity, refund address, enqueue/due times, deposit and every bridge-credit field remain durable. The Merkle leaf is always recomputed from this stored descriptor; admission atomically

writes the descriptor, deposit/escrow accounting, append frontier and root or writes nothing. Settlement reads its `dueAt` directly for the canonical head and a stored best's live boundary; there is no callback, cached deadline or duplicated scalar. Missing indices return `UINT64_MAX`. Thus `dueAt` is nondecreasing and force-due/coverage checks are one authoritative O(1) read.

The queue is a depth-32 append-only Merkle vector with fixed capacity `UINT32_MAX` items, stored root, count and 32-node append frontier. Admission rejects at capacity; an upgrade must occur before exhaustion, but already admitted items remain live. A canonical DFS range multiproof authenticates the consumed leaves and, when present, the first excluded boundary leaf under the frozen `(forceRoot,forceCutoff)`. It supplies no fully covered subtree and no unused hash; at most 129 sibling hashes authenticate a 65-leaf block range. Skipping, reordering, changing the start index or claiming an early boundary changes the reconstructed root. Kind-0 raw bytes are enqueue calldata and their hash is in the leaf; blobs are forbidden on this availability path. The deliberately unused final tree leaf avoids a nonexistent height-32 frontier slot when the old index has all 32 bits set.

Each proved block consumes the unique maximum FIFO prefix satisfying all three bounds simultaneously:

$$count \le 64, \qquad \sum byteLength \le 1{,}048{,}576, \qquad \sum accountedGas \le 20{,}000{,}000.$$

Every admitted item individually fits all three. The circuit exposes `(start,end,count, totalBytes,totalAccountedGas,forceRoot,forceCutoff,nextDueAt)` and verifies the range multiproof. Maximality means either `end=forceCutoff`, or the authenticated next leaf would exceed at least one remaining per-block cap; an early stop with a fitting leaf rejects. Candidate totals are independently capped as stated in §5. Launch enforces `ANCHOR_GAS_MAX+FORCE_GAS_BUDGET+SYSTEM_GAS_MARGIN<=L2_BLOCK_GAS_LIMIT` and the analogous witness-byte bound.

For the candidate's exact consumed interval, Settlement reads at most 256 descriptors plus the first excluded boundary descriptor when one exists under the frozen cutoff. Internal per-block boundaries are already the next consumed descriptor. It computes

```
H("slot-chain-force-descriptor-list-v2" || u64(start) ||
  u16(consumedCount) || u8(hasBoundary) ||
  (u32(index)||u8(kind)||u16(descriptorByteLength)||descriptorBytes)*)
```

as `forcedDescriptorCommitment`. The number of rows is the consumed count plus the zero-or-one boundary and is at most 257. `descriptorBytes` is the exact kind-specific packed sequence used by the leaf after its ASCII domain and index; its length is the unique constant for that kind. The circuit reconstructs every leaf, range proof, per-block maximum, candidate total and refund from these L1-authenticated descriptors. It additionally requires raw bytes for an unexpired kind 0 and the durable credit record for kind 1. An expired kind 0 needs neither historical calldata nor any unstored preimage.

Forced envelopes are auxiliary protocol inputs, not blindly copied into the Ethereum transaction list. Each block's transaction order is: (1) the exact profile-defined AnchorV4 transaction, (2) one deterministic `InboxApplyRouterV2` system transaction, (3) upfront-valid kind-0 raw transactions in FIFO order, then (4) discretionary transactions. Sequential pre-state classification puts expired, wrong-nonce, underfunded or underpriced kind-0 items only in `InboxApplyRouterV2`'s disposition vector; they have no Ethereum transaction, nonce or receipt. Upfront-valid kind 0 appears as an ordinary Ethereum transaction, whose success or revert has ordinary nonce/gas/receipt semantics. Kind 1 always produces its idempotent inbox

credit inside `InboxApplyRouterV2` and never calls the eventual destination. The system calldata commits `(forceStart,forceEnd,(queueIndex,disposition,txIndex,resultHash)*)`; the circuit derives the transaction and receipt roots from this exact order. Unused deposit is later credited to `refundAddress`; failure there cannot change consumption.

Disposition bytes are fixed: `0=EXPIRED_NO_TX`, `1=NONCE_NO_TX`, `2=FUNDS_NO_TX`, `3=FEE_NO_TX`, `4=INCLUDED_TX` and `5=BRIDGE_CREDIT`. Classification tests in that order; the first applicable no-transaction reason wins. Codes 0–3 require `txIndex=UINT32_MAX` and `resultHash=0`. Code 4 requires the exact Ethereum transaction index; `resultHash` is legacy Keccak-256 of the raw signed EIP-2718 transaction bytes. This ordinary Ethereum transaction hash is known before execution. It is deliberately not a receipt hash: putting a later receipt hash in the earlier InboxApply calldata would create a fixed point through cumulative gas. Code 5 requires `txIndex=UINT32_MAX` and this exact durable credit hash:

```
H("slot-chain-bridge-credit-result-v9" || u64(queueIndex) || creditId ||
  msgHash || u256(srcChainId) || sourceDomainId ||
  u64(srcEpoch) || address20(srcBridge) ||
  bridgeExecutionHash || u64(emittedAtBlock) || destinationDomainId ||
  u256(destChainId) || u64(enqueueBy) || address20(sender) ||
  address20(srcOwner) || address20(destOwner) || u256(value) ||
  u64(fee) || calldataHash || u8(refundMode) || address20(refundVault) ||
  refundCapsuleHash || escrowId)
```

InboxApplyRouterV2 recomputes it from the authenticated descriptor before routing the exact tuple and `resultHash` to `InboxCreditStoreV2`. That immutable store, neither Bridge nor legacy SignalService, owns the persistent destination pin. Resolver rotation and SignalService upgrades cannot retarget, fabricate or orphan an existing credit. Unknown codes or noncanonical auxiliary fields reject.

AnchorV4 and InboxApplyRouterV2 use a profile-defined, unsigned custom L2 system transaction type whose sender is injected by the post-cutover fork, not recovered from ECDSA. That type is invalid under every legacy fork, and the two reserved sender addresses are chosen with no known signing key. The circuit requires their exact profile transactions once each at indices 0 and 1 and rejects any ordinary, forced or discretionary transaction recovered from either sender. These reserved senders and the router/store pin ABI are permanent across every V2 profile; a later fork must still dispatch old routes through this router. InboxApplyRouterV2 stores the last applied L2 block number and rejects a second application in that block. An ordinary caller therefore cannot invoke the privileged path before or after cutover.

The launch fork deploys one protocol-lifetime non-proxy immutable `InboxApplyRouterV2` before every endpoint-specific immutable `InboxCreditStoreV2`. Their permanent operations are:

```
struct InboxCreditV2 {
  uint64 queueIndex; uint64 srcChainId; bytes32 sourceDomainId;
  uint64 srcEpoch; address srcBridge; bytes32 destinationDomainId;
  bytes32 msgHash; bytes32 resultHash;
}
struct InboxRowV2 {
  uint64 queueIndex; uint8 disposition; uint32 txIndex;
  bytes32 resultHash; bytes kind1Descriptor;
```

```
}
InboxApplyRouterV2.apply(uint64 forceStart, InboxRowV2[] calldata rows)
markReceivedFromInboxBatch(InboxCreditV2[] calldata rows)
    returns (bytes4 magic)
routeConfigHashV2() view returns (bytes32)
verifyInboxCredit(uint64 srcChainId, bytes32 sourceDomainId,
                  uint64 srcEpoch, address srcBridge,
                  bytes32 destinationDomainId, bytes32 msgHash)
    view returns (bytes32 creditId, uint64 processBy)
getInboxCreditSlot(bytes32 sourceDomainId, address srcBridge,
                   bytes32 destinationDomainId,
                   bytes32 creditId) pure returns (bytes32)
```

Every endpoint store constructor authorizes the same InboxApplyRouterV2; only that router may call the store batch operation, and only after that endpoint's one-shot activation. Router state includes checked `nextQueueIndex`, which the circuit requires to equal `CanonicalCore.messageCursor`. Every block calls the router once with its complete contiguous zero-to-64 forced-row interval, including kind-0 dispositions, in global queue order. It first validates the whole vector, exact result hashes, no duplicate `(domain,creditId)`, and each one-shot route `destinationDomainId->(store,Bridge,routeConfigHash,codeHash)`. Only the registrar may append a route; domain, store and Bridge are each unique and no route can be deleted or redirected. The router rechecks code/config/domain and Bridge on every use, partitions only pin side effects into maximal contiguous same-domain runs, then makes at most 64 fixed-selector, zero-value calls and requires the exact success magic. It advances `nextQueueIndex` only after all calls succeed. A missing route, conflict or later call failure reverts the cursor and every earlier store write. There is no sorting and no loop over historical routes. The batch selector is the canonical Solidity selector for the signature above. Success returndata is exactly one 32-byte ABI word equal to `0x49425632` (ASCII "IBV2") followed by 28 zero bytes; empty, short, long, malformed or different returndata rejects even if the call itself succeeds. The getter likewise must return exactly 32 bytes, and its value is

```
H("slot-chain-inbox-route-config-v1" ||
  address20(inboxApplyRouterV2) || address20(destinationBridge) ||
  address20(activationGate) || address20(terminalDomainRegistrarV2) ||
  destinationDomainId)
```

The router pins that hash and the store runtime hash at registration and rechecks both by bounded `STATICCALL/EXTCODEHASH` before any batch call. Runtime hash, not a caller assertion, pins the getter semantics. A successful no-op implementation cannot advance the cursor because the same transaction's post-state circuit must find every fresh pin at the Appendix-defined slot; the conformance corpus includes a magic-returning no-op. The canonical empty vector requires `forceStart=nextQueueIndex`, makes no store call and leaves the cursor unchanged, but still records the exact L2 block number; a second apply in that block rejects.

`kind1Descriptor` is empty for dispositions 0–4 and exactly the Appendix-defined 533-byte kind-1 descriptor for disposition 5; all other lengths reject. The router decodes that descriptor, recomputes `creditId` and the complete `bridge-credit-result-v9` hash, and derives the destination route. The circuit separately proves the same row/descriptor against the consumed L1 queue leaf. No absent owner, value, execution-hash, capsule or escrow field is treated as authoritative calldata.

Every registered store is nonproxy, unpausable, callback-free and permanently implements this exact all-or-revert pin ABI. Each run is completely validated before its first write. Activation is blocked unless the profile's system gas margin covers 64 worst-case cold route lookups/calls and 128 fresh pin writes. Each kind-1 row conservatively carries `accountedGas= MAX_FORCE_MESSAGE_GAS=5,000,000`; opcode repricing may reduce throughput but cannot leave an old head underfunded. The 20,000,000 per-block forced-gas budget therefore admits at most four kind-1 rows per block. The store validates widths and nonzero fields and performs all writes atomically. The slot is exactly

```
H("slot-chain-inbox-credit-slot-v4" || sourceDomainId ||
  address20(srcBridge) || destinationDomainId || creditId)
```

At the append-only `inboxResultHash[slot]` and `inboxProcessBy[slot]`, an empty value stores the result and `block.timestamp+BRIDGE_PROCESS_TTL_SECONDS`. The initial TTL is 2,592,000 seconds. The contract exposes no proxy, upgrade, delegatecall, mutable writer/reader/domain target, arbitrary storage primitive or pause gate. A repeated row succeeds without writes only when the stored result matches and never extends `processBy`; any conflict, ordering error or excess row reverts the whole batch before a write. `verifyInboxCredit` recomputes `creditId` and the slot, requires the received bit and nonzero result, and requires `msg.sender` to be the constructor-fixed destination Bridge for that exact local domain before returning the ID and deadline. Runtime hash, constructor router/Bridge/gate values, the one-shot sealed local-domain slot and initial empty mappings are migration-proved; the noncircular construction descriptor deliberately excludes the derived domain value. This is a new bytes32-domain ABI; it never overloads legacy `getSignalSlot(uint64,address,bytes32)`.

BridgeV2 introduces `SourceContextV2`. Its three fields are a `bytes32 sourceDomainId`, a `uint64 srcEpoch` and an `address srcBridge`. `DestinationContextV2` contains the `bytes32destinationDomainId` and permanent `addressdestBridge`. The existing send ABI remains V1; the complete additive V2/recovery ABI is:

```
sendMessageV2(Message) -> (bytes32 msgHash, Message)
sendMessageFromVaultV2(Message) -> (bytes32 msgHash, Message)
messageContextV2(bytes32 msgHash)
  -> (bytes32 creditId, SourceContextV2, DestinationContextV2)
finalizeRefundCapsuleV2(bytes32 creditId, bytes32 capsuleHash)
markQueuedV2(bytes32 creditId, uint32 queueIndex)
cancelUnqueuedMessageV2(bytes32 creditId)
finalizeDoneV2(bytes32 creditId, bytes proof)
processMessageV2(Message, SourceContextV2, DestinationContextV2)
  -> (Status, StatusReason)
retryMessageV2(Message, SourceContextV2, DestinationContextV2,
               bool isLastAttempt)
failMessageV2(Message, SourceContextV2, DestinationContextV2)
expireMessageV2(bytes32 creditId, SourceContextV2, DestinationContextV2)
recallMessageV2(bytes32 creditId, bytes proof)
withdrawSourceCreditV2(address payable recipient)
isMessageFailedV2(bytes32 creditId, TerminalProofV2 proof) -> bool
messageStatusV2(bytes32 creditId) -> Status
ActiveSettlementRouter.canonicalAt(uint64 protocolVersion,
```

```
                                    uint64 canonicalSequence)
  -> CanonicalHistoryV2
TerminalSignalVerifier.verifyTerminalSignal(
  bytes32 destinationDomainId, address destBridge,
  bytes32 creditId, uint8 terminal, TerminalProofV2 proof) -> bool
```

`CanonicalHistoryV2` is the fixed tuple

```
(uint64 protocolVersion, bytes32 executionProfileHash,
 uint64 canonicalSequence, uint48 l2BlockNumber, bytes32 blockHash,
 uint64 tipSlot, bytes32 stateRoot, uint32 messageCursor,
 bytes32 winningDataCommitment, uint256 nextBaseFee,
 uint64 nextExcessBlobGas, bytes32 terminalRoot,
 uint64 terminalCount, uint64 canonicalizedAtBlock)
```

and `TerminalProofV2` is

```
(CanonicalHistoryV2 canonical, uint64 leafIndex, bytes32[64] siblings)
```

The verifier requires the exact sequence-tagged ring entry, requires `leafIndex<terminalCount`, derives the exact domain-separated leaf from index, domain, Bridge, credit ID and terminal, and folds exactly 64 siblings to the stored root. There are no dynamic arrays, RLP nodes, current EVM state proof or trailing elements. `terminal` is exactly 1 for DONE or 2 for FAILED; all other values reject. The verifier constructor fixes only `ActiveSettlementRouter` and depth 64; it is generic across every append-only destination endpoint. Its explicit domain and Bridge arguments are low-level leaf inputs, never source authority. Each source Bridge terminal transition loads `destinationDomainId` from its immutable `CreditAuthorization`, bounded-static-reads the corresponding permanent `destBridge` from `BridgeDomainRegistry`, and passes those values. It rejects if the descriptor is absent or differs from the proof; no caller or proof chooses either value. A frozen D1 source endpoint can therefore finalize D1 and a later registered D2 without changing its verifier. Here `Message` is the existing IBridge tuple and `msgHash=hashMessage(Message)`. Destination processing first requires the context's destination Bridge to equal its own address and the context's domain to equal its one-shot sealed local destination domain. It then recomputes that domain from the pinned chain ID, genesis, BridgeInboxAdapter, active settlement router, terminal verifier, InboxApplyRouterV2, InboxCreditStoreV2, protocol-release authority, terminal-domain registrar, terminal accumulator, Bridge address/frozen execution hash, infrastructure hash and namespace. Only then do they recompute `creditId` from the Message hash and context before any status, value or external effect. No caller-supplied foreign domain may query a pin through a different funded Bridge. Source terminal functions load the immutable authorization and current permanent liability by credit ID, while destination expiration loads the permanent inbox pin; neither needs the Message preimage. Capsule finalization is authorized only to the exact recorded refund vault, and queue marking only to the adapter in the recorded destination domain. The read-only status view accepts the derived key. The source facade records both contexts. Its profile-pinned V2 event carries the credit ID, message hash, both contexts, Message, `refundMode` and `refundVault`; destination changes emit the credit ID and new status.

The destination domain's frozen Bridge facade owns the V2 status, retry state, ETH custody and terminal index for the domain's entire lifetime. Its address, runtime and storage

never rotate and it never delegates execution. This prevents another implementation or address from seeing an empty status and processing a pinned credit twice, and keeps RETRIABLE liquidity in the same account. The sole V2 admission is `verifyInboxCredit`; there is no caller-supplied nonempty source-proof path on L2. Processing rejects after `processBy`. Once that deadline passes, anyone may atomically change `NEW` or `RETRIABLE` to `FAILED` from the durable pin alone; `DONE`, `FAILED` and `RECALLED` are terminal. Before the deadline, `failMessageV2` is authorized only to `destOwner` and only from RETRIABLE. A retry with `gasLimit=0` or `isLastAttempt=true` likewise requires destOwner; no observer can force early failure. Send/process/retry/manual-fail remain pausable. In contrast, destination expiry and terminal-accumulator append, source cancellation, DONE finalization, FAILED recall, pull withdrawal and official vault refund claims are explicitly outside every guardian pause gate. Thus loss of the full Message preimage cannot lock source value forever. On transition to DONE or FAILED, the frozen destination Bridge makes exactly one call to the immutable protocol-lifetime `TerminalAccumulatorV2`. Every process, retry, fail and expire entry shares one non-reentrant guard, and arbitrary recipient execution finishes before a terminal status is installed. The Bridge then writes DONE or FAILED and `terminalIndex=UINT64_MAX`, calls only `appendTerminalV2(creditId)`, and replaces the sentinel with the returned index before returning. The accumulator accepts no terminal or domain argument. The permanent ABI is:

```
TerminalAccumulatorV2.appendTerminalV2(bytes32 creditId)
    returns (uint64 terminalIndex)
DestinationBridge.terminalCommitmentV2(bytes32 creditId)
    view returns (bytes32 destinationDomainId, address destBridge,
                  uint8 terminal, uint64 terminalIndex)
TerminalAccumulatorV2.getNodeAt(uint64 count, uint8 height,
                                uint64 nodeIndex) view returns (bytes32)
TerminalAccumulatorV2.getProofAt(uint64 count, uint64 index)
    view returns (bytes32[64])
```

All selectors are canonical Solidity selectors of those signatures. Append returndata is exactly 32 bytes with a zero-padded `uint64`; commitment returndata is exactly four ABI words (128 bytes), with canonical high padding, `terminal=1` for DONE or 2 for FAILED and `terminalIndex=UINT64_MAX`. Node and proof returndata are exactly 32 and 2,048 bytes respectively. Short, trailing, noncanonical, overflow, revert or out-of-gas returndata rejects atomically. The accumulator uses exactly 50,000 gas for the commitment `STATICCALL`; the release gas benchmark must show 30% margin or revise this normative constant and the profile before launch. With that fixed gas stipend and exact return length it `STATICCALL`s the registered caller's `terminalCommitmentV2(creditId)`, requires the sealed domain, caller address, DONE/FAILED status and sentinel, then appends:

```
leaf = H("slot-chain-terminal-leaf-v1" || u64(terminalIndex) ||
         destinationDomainId || address20(destBridge) ||
         creditId || u8(terminal))
```

Any call or return failure reverts status, sentinel, leaf and count together. During a V1 recipient callback there is no concurrent sentinel, so a message targeting the accumulator cannot forge a leaf even though `msg.sender` is the Bridge. V2 additionally rejects the accumulator address as a message target.

The accumulator stores a checked `uint64count`, root, every immutable leaf and every fixed subtree node exactly once when that interval becomes complete. Append stores the leaf plus `trailingOnes(terminalIndex)` newly completed ancestors and computes the root from those nodes and canonical empties; it never overwrites a completed interval. `getNodeAt(count, height,index)` returns the canonical empty node when the interval begins at or after `count`, the stored completed node when the interval ends at or before `count`, and otherwise recursively hashes the unique partial boundary. Thus `getProofAt(count,index)` reconstructs exactly 64 siblings for any historic count still named by Settlement, even if the accumulator has since advanced. At count N, fewer than $2N + 65$ storage slots are occupied; one append performs at most 65 hashes and at most 65 writes, with fewer than two new completed-node writes amortized. A missing completed node for an occupied interval is an invariant violation and never decodes as empty. Append rejects at `count=UINT64_MAX`; the unreachable final index preserves the sentinel.

Registration is append-only. Two protocol-lifetime non-proxy contracts are:

```
ProtocolReleaseAuthorityV2
TerminalDomainRegistrarV2
```

Only the registrar may register an accumulator writer. Release authority is bound to the fork-reserved system sender and manifest namespace, not a particular Anchor version or endpoint. Activation passes the value, never merely its hash:

```
struct ComponentDescriptorV2 {
  address component; bytes32 runtimeHash; bytes32 configHash;
}
struct ReleaseManifestV2 {
  uint64 protocolVersion;
  uint256 settlementChainId; uint256 destinationChainId;
  bytes32 destinationGenesisHash; bytes32 executionProfileHash;
  bytes32 manifestNamespace;
  address anchorV4; bytes32 anchorRuntimeHash;
  address activationGate; bytes32 activationGateRuntimeHash;
  bytes32 destinationDomainId; address destinationBridge;
  bytes32 destinationBridgeExecutionHash;
  bytes32 destinationInfrastructureHash;
  ComponentDescriptorV2[9] components;
}
AnchorV4.activateReleaseV2(ReleaseManifestV2 calldata manifest,
                           bytes calldata mptProofV1)
ProtocolReleaseAuthorityV2.activateRelease(
    ReleaseManifestV2 calldata manifest)
TerminalDomainRegistrarV2.activateDomain(
    ReleaseManifestV2 calldata manifest)
```

The manifest tuple's static ABI body is exactly 41 words/1,312 bytes with canonical address and integer padding and no offsets. Its canonical commitment encoding is the 1,144-byte packed concatenation of the fields above, in order, using `u64`, `u256`, `address20` and `bytes32`, followed by each component's address/runtime/config tuple. It is

```
releaseManifestHash = H("slot-chain-release-manifest-v1" || u32(1144) ||
                        canonicalReleaseManifestV2)
```

Zero values, duplicate component addresses, a Bridge different from component 9, a component count/order mismatch, or any named/hash field inconsistent with the Appendix grammars rejects.

The AnchorV4 custom system transaction invokes that exact selector with the complete manifest plus one `MptProofV1`, under the same 132-node, 80,000-byte and 8,000,000-gas limits above. The circuit authenticates the transaction's exact `l1OriginStateRoot` as the finalized L1 origin for this L2 block. Anchor verifies under that root the profile-pinned L1 ProtocolVersionManager account, runtime code hash and the exact `releaseManifestHash[protocolVersion]` storage word; it recomputes the hash from calldata and never accepts a proof-valid Boolean. That L1 mapping's base, element slot and storage-trie key are:

```
base = H("slot-chain-protocol-version-manager.releaseManifestHash.v1")
elementSlot = H(bytes32(uint256(protocolVersion)) || base)
storageTrieKey = H(elementSlot)
```

Its nonzero value uses canonical minimal-unsigned RLP. Anchor then stores and exposes `activeReleaseManifestHashV2()viewreturns(bytes32)` with exact 32-byte returndata, and in the same non-reentrant transaction calls the authority and registrar with the identical fixed manifest. Partial completion reverts the active hash and both callees.

The authority recomputes the commitment, requires `msg.sender` and its `EXTCODEHASH` to equal `anchorV4/anchorRuntimeHash`, requires `tx.origin` to be the fork-reserved Anchor-system sender, and makes a 50,000-gas `STATICCALL` requiring Anchor's exact active-manifest getter to return the same hash. It then permanently records `releaseManifestHash[protocolVersion]` and exposes `releaseManifestHashV2(uint64)viewreturns(bytes32)` with exact 32-byte returndata. A direct call by that reserved sender, an EOA, Bridge or an Anchor from another manifest rejects: EVM `CALL` does not preserve the transaction sender as `msg.sender`. The registrar constructor fixes only that authority, the global InboxApplyRouterV2 and the accumulator. Its non-reentrant `activateDomain(manifest)` recomputes the same hash, requires the authority's exact stored word, extracts every endpoint target from those bytes, and recomputes all nine runtime/config descriptors, the separately derived destination Bridge execution hash, infrastructure hash and destination domain. Components 1–3 are L1 contracts: the ProtocolVersionManager validated their exact addresses, runtime/config hashes before storing the proved manifest hash; the L2 registrar never pretends to execute `EXTCODEHASH` across a chain boundary. Components 4–9 are L2 contracts, so the registrar directly checks each local target's exact `EXTCODEHASH`, exact 32-byte `componentConfigHashV2()` return, constructor config and zero prestate. It then makes only the profile-listed fixed-selector, zero-value, gas-bounded seal calls to the target InboxCreditStoreV2, activation gate and frozen destination Bridge, atomically adds the one-to-one inbox route, registers the domain/Bridge writer in the accumulator, burns every endpoint sealer and only then stores the single Appendix-defined `registrationCommitment[protocolVersion]` word. The version, domain and Bridge are each one-shot; a conflict, out-of-order call, unapproved descriptor, reentrancy or partial seal reverts the transaction. The same path is used for launch and later endpoints. A domain or Bridge can therefore be registered once, never deleted or redirected, and a new endpoint/domain cannot disable an old writer. The accumulator is a non-proxy immutable, unpausable contract with no public registrar, arbitrary root setter or external call except the authenticated registrar and the bounded static read of a registered Bridge during append. It

returns `terminalIndex`; the Bridge stores that index with its DONE/FAILED status and emits it. Status, accumulator append and index are one atomic transition. The accumulator leaf sequence, not a versioned EVM storage proof, is the permanent recovery statement. Every future Settlement/profile migration must import the exact prior root/count, authenticate the same accumulator account and prove root/count induction from that base; reset, fork or replacement is invalid while any V2 domain exists.

Every execution proof additionally authenticates the accumulator account and proves that the candidate's `terminalRoot` and `terminalCount` equal its exact post-state root/count slots. These fields are part of `CanonicalCore` and `executionOutputsCommitment`; tier-2 and tier-3 use the same public-output constraint. A caller cannot supply an unconstrained root; no checkpoint publisher exists in this revision. The same proof authenticates the protocol-lifetime InboxApplyRouterV2 account and requires its post-state `nextQueueIndex` to equal `CanonicalCore.messageCursor`; migration proves the same equality.

The vector root is

```
H("slot-chain-terminal-root-v1" || u64(terminalCount) || treeRoot)
```

where the Appendix fixes empty nodes, node hashing and bit order.

Every versioned Settlement is a permanent non-proxy contract with no `SELFDESTRUCT`, delegate target, history pause or code upgrade. Its internally written history cell contains the exact sequence plus the full `CanonicalCore` and `canonicalizedAtBlock`; a read requires exact tag equality. The protocol-lifetime non-proxy `ActiveSettlementRouter` keeps an append-only mapping from protocol version to exact Settlement address, `EXTCODEHASH` and execution-profile hash. Historical entries cannot be deleted, redirected or reactivated. Its only switch is:

```
activateVersion(protocolVersion, executionProfileHash,
                settlement, importProof)
```

It is callable by the delayed ProtocolVersionManager while activating that exact manifest. It requires a strictly increasing version, approved Settlement runtime/profile and one atomic migration-coordinator transaction. That transaction irreversibly freezes the old Settlement, reads its exact full `Canonical`, canonical sequence and `lastCanonicalCommitBlock`, and requires the new empty PREACTIVE Settlement to import every `CanonicalCore` field and `canonicalizedAtBlock` without narrowing. Both versions use the same protocol-lifetime ForcedQueue address; the coordinator checks its exact root, count, frontier, cursor, escrow balance and `lastDueAt`, rather than copying scalars into a replacement queue. It also derives that no normal best, recovery episode, live data session, unclaimed reward receipt, builder escrow or other transient reservation remains, or completes its exact refund/claim path before freezing. The new Settlement writes the imported full core at the same sequence in its own version-namespaced ring, sets `lastHistoryWriteBlock=block.number`, and only then becomes ACTIVE. Its first later canonical commit requires a strictly greater L1 block. Failure of any step reverts the freeze, import and switch together.

The router's stable `syncAndAppend` forced-ingress facade remains available to every old immutable BridgeInboxAdapter. It resolves the active version, calls that Settlement's bounded `sync()`, preserves a SYNCED transition without appending, and otherwise appends only to the singleton ForcedQueue. No Settlement calls the router while committing canonical state. Router `canonicalAt(protocolVersion,canonicalSequence)` performs a bounded

fixed-selector `STATICCALL` to the permanently registered Settlement, requires the exact return length, code hash, version/profile and cell tag, and returns no caller fields. This gives target replacement the same delayed trust root as the validity verifier without adding a canonical-path dependency.

The non-proxy, unpausable `TerminalSignalVerifier` accepts only the fixed 64-sibling proof against one exact routed tuple and the exact domain/Bridge/credit/terminal leaf. Source finalize/recall verifies and mutates the source credit in one transaction; there is no publication stage to front-run. Sparse 4,096-block L2 jumps cannot choose a cell, and at most one canonical write per L1 block gives a live version's 256 cells at least 3,072 seconds of retention. Frozen versions retain their last 256 cells forever. The Bridge stores `terminalIndex[creditId]`; the index and accumulator `getProofAt(canonical.terminalCount,index)` reconstruct the exact proof from current immutable completed-node state, without a historical receipt or privileged indexer. Source terminal functions never call legacy SignalService. No V2 failure, recall, delivery finalization or view falls back to `msgHash`. The release profile pins every selector, event, status, liability and accumulator slot plus positive and negative lifecycle vectors.

Official ERC20, ERC721 and ERC1155 vaults add an unversioned `RefundCapsule[creditId]` before V2 activation. ERC20 stores owner, canonical/local asset, release-or-mint mode and actual amount; ERC721 stores the same descriptor and at most 256 token IDs; ERC1155 stores at most 128 ID/amount pairs. The profile fixes canonical encoding, caps and storage slots. The vault re-encodes the destination call and requires its hash to equal the source record's `calldataHash` before registering `refundCapsuleHash`. It also reserves pooled ERC20/ERC1155 balances and exact ERC721 IDs against every live capsule; ordinary and reverse payouts cannot consume those reserves. After CANCELLED or RECALLED, only `srcOwner` or its explicit delegate may call `claimRefundV2(creditId,recipient)`. The unpausable function loads the exact permanent source Bridge status, marks the capsule claimed, then releases or mints the stored assets. A failing token transfer rolls back only that claim, so the owner may retry to a compatible recipient; Bridge terminal state and ETH pull credit are unaffected. DELIVERED makes the capsule and its reserve permissionlessly releasable. No synchronous legacy callback appears in a V2 terminal transition. V2 source pull withdrawals and capsule refund claims restore an already reserved source position: they bypass every ordinary Bridge Ether quota and vault token quota, decrement only the exact liability or reserve under checks-effects-interactions, and cannot draw from unreserved liquidity. V1 processing and callback outflows remain quota-limited and must also leave every V2 reserve intact. The vault facade and capsule storage are frozen at cutover; no later delegatecall implementation may reinterpret or release an old reserve. Burn/mint-mode V2 sends additionally require the source bridged token's profile-frozen `IRefundRestorableV2` implementation. Its unpausable `restoreRefundV2(creditId,recipient,amountOrIds)` accepts only the exact frozen vault, consumes each credit ID once and bypasses ordinary mint quota. An old bridged token without that runtime stays on V1. For a locked externally governed canonical token, the capsule remains claimable but the actual transfer is conditional on that token's own liveness as stated in §1.

The L2 cutover latch enables V2 processing only after atomic registrar sealing. Source V2 selectors remain disabled until that canonical registration is proven to L1 and its 214-block support delay completes. Every old `sendMessage` call and all L2-to-L1 sends retain V1 process/retry/fail/recall. InboxApplyRouterV2 makes only the bounded manifest-routed store calls above, and neither router nor store invokes a message recipient. Legacy proof-based `proveSignalReceived` remains unchanged for V1 and is never a V2 terminal dependency. An-

chorV4 and InboxApplyRouterV2 are profile-defined system transactions: their custom type, reserved senders, system nonces, zero beneficiary tip and fee treatment are consensus fork rules, not ordinary user accounts. Their gas and the batch loop still count against the block limit. Deployment and migration require vector-tested InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2 and TerminalAccumulatorV2 bytecode. Legacy SignalService remains V1-only and may be upgraded without entering a V2 trust path. Authorizing an unreviewed caller is invalid.

An unexpired kind-0 item must reveal raw bytes hashing to its stored `rawTxHash`; the circuit decodes them and matches every duplicated descriptor field before execution. Once `validUntil` is strictly before the L2 block timestamp, the circuit emits the unique `EXPIRED_NO_TX` disposition from the authenticated durable descriptor without needing those bytes. Consequently a disappeared sender or pruned history can delay its own item only until the bounded validity horizon, never wedge the FIFO permanently. A user that wants execution remains an available source for its own signed bytes. The 2,164-second recovery target is conditional on unexpired head bytes being retrievable; the unconditional protocol bound adds at most `MAX_FORCE_VALIDITY_SECONDS`.

For a normal signed block, `forceRoot/forceCutoff` are authenticated by its L1 anchor. For recovery they equal the current round's directly stored pair. The circuit consumes only up to that cutoff, so a later append cannot invalidate an already signed block or in-flight proof. All blocks in one candidate use the same cutoff. A due current head cannot be bypassed: `sync()` cancels an immature window, or closes a mature best only when its live boundary is not due, before opening recovery.

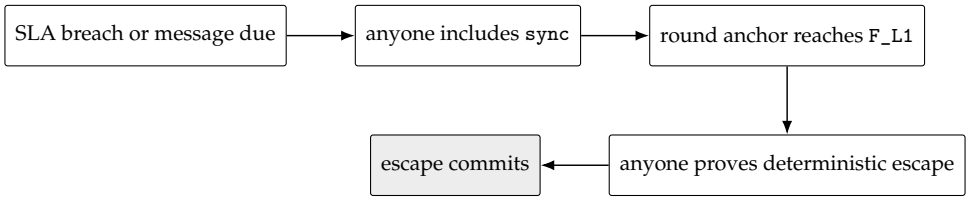
The 1,500-second force delay applies to the queue head, not to an arbitrary position behind existing prepaid work. An admitted sender can compute its worst-case number of escape blocks from the gas ahead at admission. As with L1 itself, the protocol does not promise backlog-independent latency against an adversary willing to buy all available capacity; it promises that builders cannot selectively veto the FIFO head.

An escape block is deterministic under a governance-frozen `executionProfileHash`. That profile commits the L2 fork rules; complete header-field derivation (gas limit, base fee, extra data, withdrawals/requests roots and randomness); the deployed AnchorV4, InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2 and TerminalAccumulatorV2 bytecode/constructor hashes; reserved senders, nonces and gas; AnchorV4 checkpoint calldata and `ancestorsHash` transition; and the transaction/receipt derivation above. For the initial profile the anchor is an EIP-1559 transaction from:

`0x0000777735367b36bC9B61C50022d9D0700dB4Ec`

Its gas limit is 1,000,000, priority fee/value zero, max fee equal to the block base fee, empty access list, and the sender's sequential state nonce. It calls the frozen Anchor address with selector `0x523e6854` and ABI tuple `(uint48(anchorNumber),anchorHash,anchorStateRoot)` from the candidate's one batch anchor. Its deterministic protocol signature algorithm and the InboxApplyRouterV2 transaction's corresponding fields are part of the profile vectors. Its timestamp is `GENESIS_TIMESTAMP+escapeSlot`, coinbase is the protocol sink and there is no discretionary body. A repeated batch anchor still performs the exact per-block ancestors transition. Changing the execution profile requires a delayed protocol-version upgrade and new full-block golden vectors; runtime guessing is forbidden.

For a cartel with no usable builder block, the recovery sequence is:



With the initial bounds, activation inclusion (120 s), the frozen `ESCAPE_OFFSET=1,900s` (covering `T_DEPTH_MAX=900s`, proving at 900 s and 100 s margin), submission inclusion (120 s), and clock margin (24 s) total 2,164 seconds, below `DELTA_FINAL_LAG=3,600s`. Tip admissibility is separate: the escape slot is at the end of the offset, so only submission and skew—144 seconds—must fit `DELTA_TIP=1,200s` when proof generation meets its bound.

9 Economics, slashing and payments

Event	Objective evidence	Effect
Same-context equivocation	Two distinct protocol-domain signatures for one key/slot/context	Slash that schedule window's tranche once; tombstone key; emit reporter entitlement.
Aggregator SLA breach	Final lag exceeds threshold after mature close	Terminate and burn penalty bond once; open recovery; promote standby if any.
Force-only activation	Due queue head	No aggregator penalty; open recovery.
Skip or absence	None that distinguishes fault from non-receipt	Not slashable; promise may be revoked.
Invalid proof/data	Verifier failure	Reject; no canonical or payment effect.

Equivocation evidence is idempotent per `(builder,window)`; separate windows slash separate tranches. `L_LEASE` is calibrated against at most 76 assigned slots but is not insurance for unbounded signed value. Tombstoning is atomic with the first valid slash; future snapshots exclude the key.

Evidence does not depend on retaining an obsolete `admissionRoot`. It contains two distinct low-s EIP-712 headers with the same recovered nonzero key, slot, tier, context ID, admission version, admission root, episode, revision and recovery ID. For tier 1 it recomputes the normal context from the same base, admission pair and arm-block anchor carried by both headers and authenticates that arm hash as canonical through EIP-2935; tier 2 recomputes the recovery context. The contract enforces at launch that the last tier-1 evidence/replay instant precedes expiry of that 8,191-block history entry. It supplies one proof under the signed admission root and one under the *current* admission root for the same address and `registrationIndex`; the latter must be that still-retained active/liability generation. Its current stored tranche root separately authenticates the window leaf, which is `RESERVED(W)` or `LIABLE(W)`. The two complete struct hashes must differ. Signing two conflicting protocol headers in one such context is slashable even if the key was not ultimately selected at that schedule position; builders must not sign off-ticket headers. The evidence transaction derives W from the slot, requires the tranche leaf's stored window to equal W, and lands by `evidenceReplayDeadline(W)=evidenceDeadline(W)+REORG_MARGIN_SECONDS`. Evidence available by `evidenceDeadline(W)` therefore has the full

margin for reorg detection and re-inclusion; first publication delayed into the replay tail has no replay guarantee. Address reuse is forbidden while that retained generation exists, so the current proof cannot select a newer incarnation. This rule keeps evidence verification fixed-depth while the liability-capacity invariant retains every slashable generation through the replay deadline.

Canonical settlement performs no reward calculation, token balance write or external call. It always emits a candidate-committed event containing the ID, beneficiary and reward class. Best-effort protocol claims use a fixed `MAX_REWARD_RECEIPTS=256` ring, selecting `uint8(uint256(candidateId))`: if that cell is empty or its prior receipt is both claimed/expired and past the reorg margin, Settlement records the three fields, commit block and `claimed=false`; otherwise it only emits the event. Ring occupancy can therefore forfeit a reward but can never revert or delay canonical state. A separate distributor verifies this stored receipt, sets `claimed=true` before transfer, and pays at most the funded class cap before `REWARD_CLAIM_WINDOW=86,400s`. Logs alone are explicitly not on-chain-verifiable entitlements. Burns are never recycled into the same episode's bounty. Normal service/data compensation likewise cannot affect candidate validity or canonical commit.

10 Security and liveness analysis

Adversary/failure	**Protocol result**	**Bound or residual**
Invalid execution or forged state	Proof rejects.	Finalized-state safety reduces to proof-system and L1 safety.
Aggregator offline/byzantine	SLA activation terminates it; any prover uses tier 2 or 3.	At most final-lag threshold plus 2,164 s under stated inclusion/proof and head-data bounds.
No seat or no standby	State remains valid; escape has no seat dependency.	Same protocol bound; reward may be zero.
All builders collude/absent	Unsigned escape advances anchor and forced queue.	Meaningful discretionary throughput falls to forced envelopes; no builder censorship veto.
Builder equivocation	Competing signed branches; one may win normal order.	Direct per-window slash; preclose promises remain revocable.
Builder skips/withholds body	Honest tail may be unprovable or omitted.	Not objectively slashable; eventual escape restores state progress but may revoke promises.
Data-session spam	Sessions are publisher-owned and bonded.	Attacker fills only its session; escape uses no blob session.
Missing/pruned kind-0 bytes	Unexpired head waits; after validity it is consumed as EXPIRED_NO_TX from its durable descriptor.	Adds at most seven days; a kind-1 descriptor is durable, while successful destination invocation still needs Message bytes.

Bridge or vault upgrade exploit	V2 custody/recovery facades are frozen and never delegatecall another implementation.	A new endpoint/domain cannot redirect an old writer; every old endpoint appends to the protocol-lifetime terminal vector.
Unauthorized source-support fill	Version manager rejects entries absent from the active delayed manifest.	At most 64 audited additions per release; no attacker-consumable lifetime cap.
Message bytes disappear	Enqueue expires to source cancellation; a pinned credit expires to destination FAILED.	ETH pull credit and bounded official-vault capsules make refund permissionless by credit ID; successful destination invocation is lost.
Pooled source liquidity drains	Every V1/V2 payout and sweep checks post-balance against live ETH liability; vault payouts check fungible or exact-NFT reserves.	QUEUED value releases only after proved DONE or as a pull refund after proved FAILED.
Guardian pause or zero quota	Risky sends/process/retries stop; expiry, terminal verification and reserved refunds continue.	Frozen V2 proof/refund paths have no transitive pause or ordinary quota dependency.
Foreign destination replay	Local domain and `destBridge==address(this)` checks reject before status or value effects.	The immutable pin store and accumulator both bind one exact domain/Bridge pair.
Terminal proof-format change	Canonical state carries a stable application-level depth-64 vector root/count.	Source verification does not parse the destination EVM state format; later roots preserve old leaves.
Legacy call to installed V2 code	Shared L2 gate is false and the custom system transaction type is invalid.	First proved post-cutover Anchor activates once; all V2 modules recheck the gate.
All canonical prestate copies lost	No party can construct the next EVM witness from a root alone; proof generation stops without changing canonical state.	Explicit information-theoretic availability limit; restore any root-verifiable archive copy.
Transparent-mempool front-run	First valid recovery transaction wins.	Proof statement binds beneficiary, episode, revision and outputs; copying cannot redirect payment or state.
Payment-vault exhaustion	Claim is zero or partial.	Never blocks safety or canonical liveness; economic liveness is conditional.
L1 reorg within policy	Contract state, burns and claims roll back together.	Clients replay canonical logs and proofs against new version/hash.
L1 censorship/outage	Neither activation nor proof can land.	Outside model after `T_INCLUDE_MAX_SECONDS`; no L2 protocol can force its L1 contract to execute.

The escape lane establishes strong transaction-entry permissionlessness even when the capital-ranked builder set is captured. It does not make the normal builder market egalitarian: top-64 ranking and per-window capital remain a meaningful entry barrier. The product must distinguish these two claims.

11 Implementation blueprint

11.1 L1 modules and bounded storage

- `BuilderRegistry`: 64 active cells, 1,072 liability generations, 512-leaf tranche roots, tombstones, replacement counters and the versioned `admissionRoot`. Every transition is fixed-depth or constant-size.
- `ScheduleOracle`: sealed window root/seed in a ring covering `MAX_LIVE_WINDOWS=268`. `MAX_EARLY_SEAL_WINDOWS=8` covers the earliest snapshot geometry. In window units the capacity is at least:

  ```
  MAX_EARLY_SEAL_WINDOWS + MAX_CANDIDATE_WINDOWS +
  ceil((W_SETTLE_SECONDS + DELTA_FINAL_LAG + DELTA_TIP +
        REORG_MARGIN_SECONDS + EVIDENCE_DELAY_SECONDS) / 384) + 2
  ```

  An entry is overwritten only after it is `EXPIRED`; the same transaction checks the release predicate and no stored normal best/round reference.
- `DataSession`: a 1,024-cell global ring, at most two live cells per owner, each with owner/nonce ID, 2,100-leaf MMR frontier, sealed root/count and expiry; no on-chain leaf array. Expired eviction advances at most eight cells per call.
- `ForcedQueue`: depth-32 append-only Merkle root/count/frontier, last due time, complete fixed-width per-index descriptors and deposits; an expired kind-0 descriptor reconstructs its leaf, prefix accounting and refund without historical calldata, while kind-1 credits remain durable. Only the router appends.
- `BridgeDomainRegistry`: non-proxy append-only source, destination and frozen-endpoint-support mappings. Only bounded entries from the manifest being atomically activated by ProtocolVersionManager may be added, and destination support additionally requires a finalized canonical L2 proof of the exact registrar record; there is no delete, redirect or age-proportional iteration.
- `BridgeInboxAdapter`: exactly-once NONE/QUEUED records and caller-funded queue deposits. It direct-reads same-L1 CreditRecord and Bridge liability state, rejects while PRE-ACTIVE and, after router SYNCED, commits that transition, creates no record and pull-credits the full caller deposit before a nonreverting return. A successful append atomically marks the permanent source record QUEUED. It is non-proxy and immutable; its runtime and constructor-fixed registry, Bridge, router and version manager plus its one-shot sealed destination-domain slot are profile/domain-bound, with no mutable target, delegatecall or arbitrary-call selector.
- `InboxApplyRouterV2` and `InboxCreditStoreV2`: the router is a protocol-lifetime non-proxy dispatcher with one global cursor and append-only registrar-authenticated domain routes; every endpoint store fixes that same router and its own Bridge/domain. Full-vector validation, at most 64 contiguous-run calls and EVM rollback make mixed-domain application atomic. Neither has an upgrade, resolver, callback or caller-selected target.

- `ProtocolReleaseAuthorityV2`: protocol-lifetime non-proxy release manifest authenticator for the manifest-derived Anchor caller, permanent reserved system-transaction origin and namespace. Its version records are one-shot and have no endpoint target.
- `TerminalDomainRegistrarV2`: non-proxy immutable version-activation coordinator bound only to the release authority and accumulator. It derives endpoint targets from the authenticated manifest and atomically seals the store, Bridge, activation gate, inbox route and accumulator writer. Records are permanent, one-shot and one-to-one.
- `TerminalAccumulatorV2`: protocol-lifetime non-proxy depth-64 append-only vector with checked count/root, immutable leaves/completed subtrees, `getProofAt(count,index)`, one-to-one domain/Bridge writer registration and no root setter. Its sentinel/static-read handshake derives the terminal from the Bridge; each transition stores the returned index atomically.
- Permanent source/destination `BridgeV2Facade` endpoints retain custody, aggregate/per-credit liabilities, status, pull credits and signal-app identity. Their old proxy upgrade surface is disabled, their implementation slot and runtime are migration-proved, and they never delegatecall mutable code.
- Official token vaults retain bounded refund capsules, reservation totals or exact NFT IDs and claimed bits. Their unpausable V2 claim path has no Bridge callback, quota check or loop beyond the profile's 256-word capsule bound; the vault implementation is frozen with the Bridge. Burn/mint mode accepts only a profile-frozen bridged-token runtime with the exact once-per-credit unpausable restoration selector.
- `Settlement`: permanent non-proxy state machine with `PREACTIVE`, `ACTIVE`, `MIGRATION_ARMED`, `MIGRATION_READY` and `FROZEN` states; canonical core, one normal best, recovery episode/round, seat pointer, 256-cell sequence-tagged full-core history and a 256-cell best-effort receipt ring. A delayed cutover stops opening new work and reaches MIGRATION_READY after the current transition settles, so an adversary cannot veto migration by continuously creating a best or recovery round. Rewards and burn disposal remain separate; no loop is proportional to chain age.
- `ActiveSettlementRouter`: non-proxy, no generic target setter. Its append-only version registry permanently binds Settlement address/code/profile; activation imports the exact full core, same ForcedQueue and migration-ready state. Its stable sync/append facade preserves old adapters, while `canonicalAt(version,sequence)` only reads exact historical cells.
- Terminal verifier: immutable, read-only and unpausable; it checks one fixed 64-sibling vector proof against an exact router-routed version and sequence cell. The leaf separately binds index, domain, Bridge, credit and terminal status.

The non-authoritative `CanonicalArchive` is independently mirrored off-chain canonical bodies, receipts, state-trie nodes and runtime bytecode indexed by canonical block, state root and code hash. Every object is commitment-verifiable; no archive address, signature or availability vote appears in Settlement or the verifier statement.

All setters enforce parameter inequalities atomically and are disabled after the launch freeze except through the existing delayed governance path. Every counter has an explicit width and checked increment; no sentinel shares a legal value.

11.2 Historical authentication

Native `blockhash` plus a supplied matching RLP header authenticates a fresh normal arm block; EIP-2935 authenticates later normal-context evidence and schedule carrier headers. A recovery round captures its immediately preceding block hash with `blockhash(block.number-1)` and later hashes a supplied RLP header to that stored value; it never tries to read an unavailable parent timestamp, queries an expired history entry or mutates the meaning of a past header. Its queue pair is round storage, not an MPT claim. EIP-4788 plus SSZ branches authenticates the schedule's parent beacon/execution payload; its state root plus MPT proofs authenticates the registry. Deployments lacking either primitive require an equivalently verified oracle and separate review.

11.3 Executable execution profile

`executionProfileHash` is not a free-form governance label. Each `protocolVersion` maps immutably to one canonical CBOR profile whose bytes are published with the verifier key and hashed as `H("slot-chain-execution-profile-v1"||u32(byteLength)||profileBytes)`. The decoder rejects unknown keys, duplicate map keys, nonminimal integers, indefinite lengths and zero code hashes. The profile contains:

- settlement/L2 chain IDs, both genesis hashes, fork digest, EIP-2935 address and activation block; BridgeDomainRegistry/ProtocolVersionManager addresses and runtime hashes; every append-only source, destination and support descriptor and manifest binding; each nonzero domain namespace, frozen Bridge/vault facade and credit-registry runtime, implementation slot, terminal verifier, immutable settlement router, BridgeInboxAdapter, InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2 and TerminalAccumulatorV2 runtime plus every constructor immutable, infrastructure hash, ABI, CreditRecord/liability/status/index layouts and direct-call gas limit;
- gas limit, EIP-1559 elasticity/change denominator, blob target/update fraction, empty ommers/withdrawals/requests constants and exact derivation of base fee, blob fields, logs bloom, randomness, parent beacon root, requests hash and every post-fork header field;
- the Anchor, V2 activation gate, InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2, TerminalAccumulatorV2 and both legacy SignalService contracts, permanent source/destination Bridge facades, official refund vaults and frozen bridged-token restoration template addresses and runtime code hashes, every expanded V1/V2 selector and ABI tuple; the unsigned custom system transaction type and unforgeable reserved senders; system nonce rules, fee exception, activation latch and gas ceilings;
- the exact AnchorV4 checkpoint/ancestors transition, forced disposition rules, InboxApplyRouterV2 global cursor, route and batch transition, immutable store inbox slot, result and deadline storage, Bridge V2 aggregate and per-credit liability, both endpoint statuses, terminal indices, vector count, root and immutable completed-node store, pull credits, refund capsule, reservation and claim layouts, cancellation, expiry, delivery and recall transitions and the sole inbox-pin path; and
- at least six complete vectors, each covering parent, prestate, input, transaction, receipt, header and poststate: empty escape, valid kind 0, expired kind 0 without bytes, kind 1 under two separately registered frozen source generations, source and destination do-

main replacement, direct-record absence and mismatch, enqueue and cancel ordering, router-SYNCED refund and retry, capacity race, delayed append, PREACTIVE kind 0 and kind 1 rejection, legacy attempts to invoke inbound V2, first post-cutover latch activation, pin survival across arbitrary legacy SignalService upgrades, endpoint-support manifest authorization/finality and finalized L2 registration proof, squatting/unauthorized/out-of-order registration, mixed D1/D2/D1 routing, old-domain enqueue after new-domain activation, missing-route atomic rollback, multi-credit batch ordering/conflict, Message loss before enqueue and after pin, deadline expiration, DONE/FAILED terminal replay and a cap-limited multi-block prefix, plus extra-reserved-sender, unauthorized clone, mismatched/local-foreign domains, capsule absence/hash mismatch/double claim, pooled-balance drain attempts, zero ordinary quota during V2 refund, legacy SignalService pause during V2 terminal proof, forbidden upgrade/delegatecall, exact legacy-send compatibility, explicit direct/vault V2 selectors, old-domain accumulator append after new-domain activation, canonical-sequence cell collisions, accumulator-target caller confusion, sentinel/wrong-credit/duplicate-append, terminal leaf/index/root substitution, historic-count proof reconstruction, unauthorized/fake/full-core-discontinuous router switches, same-block history writes, migration-ready anti-griefing, conflicting result/terminal hash and every V1/V2 boundary.

For escape, parent hash/height/state and next fee/blob scalars come from the stored canonical core; timestamp/slot, L1 origin and forced inputs come from the round. Every remaining header field and both system transactions are a pure profile function of those values and the execution result. The circuit loads the profile selected by the public protocol version and proves its hash equals Settlement's immutable mapping. Launch tooling rejects a version lacking the CBOR artifact, verifier-key binding or complete vectors. Thus a profile change is a new protocol version and review, not runtime interpretation.

11.4 Reorg rule

The canonical L1 log is the journal. A reorg truncates and replays, in order: canonical tuple, mode/version, normal arm/activation/best/deadline/frozen admission, episode/round, seat termination/burn, tombstone/tranche state, forced cursor, data-session root/count, and reward eligibility. Off-chain workers index by `(settlementChainId,contract,blockHash,logIndex)` and discard orphaned jobs. Withdrawal/bridge execution waits for the configured L1 finality policy.

11.5 Genesis and migration

Deployment occurs at least `D_SNAP+F_FINAL+sealMargin` before activation, so first live sources are post-deployment and sealable. New Settlement begins `PREACTIVE`; registry, scheduling and data preparation may run, but every candidate and every kind-0/kind-1 forced-ingress or bridge enqueue call rejects before creating a deposit, adapter record or queue leaf.

The legacy Inbox does not share this design's queue or expose `freezeAndExport`. Migration therefore never pretends to adopt its blob-backed forced-inclusion records. A staged legacy upgrade first disables new legacy forced admission, then drains every admitted legacy record only through the legacy protocol's native consume/expiry/void rules. Existing records do not identify a general refund owner, so this design promises neither a fabricated refund nor conversion. Any separately governed claim mechanism must complete before migra-

tion and is outside this adapter. Activation is blocked until the audited native legacy predicates report an empty live queue and zero unsettled forced escrow. Only then is the new empty `ForcedQueue` placed behind `ActiveSettlementRouter`; its initial root/count/cursor and `dueAt(0)=UINT64_MAX` are golden migration values. Existing Taiko bridge messages remain on the legacy SignalService/Bridge retry path and are not silently converted into kind 1. Only new, explicitly escrowed inbox-credit messages use `BridgeInboxAdapter`; its direct-read descriptor binds the existing message hash, both domains, source epoch, permanent Bridge/execution identity, source emission block, enqueue deadline, sender/refund vault, capsule hash, fee and ownership/value/calldata fields, while destination invocation remains a later bridge operation.

After that drain, migration has three phases. First, before quiescence, the legacy L2 governance path deploys the profile-exact protocol-lifetime non-proxy InboxApplyRouterV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2 and TerminalAccumulatorV2, then the endpoint InboxCreditStoreV2. The global router's constructor fixes only the permanent system sender, registrar and namespace. The store's constructor fixes that router, Bridge, activation gate and registrar, but its local-domain slot remains zero; the Bridge's corresponding slot and accumulator registration also remain empty. All component addresses are precomputed from the deployment coordinator's fixed CREATE nonce sequence recorded in the manifest, so constructor references do not require an address/init-code fixed point. It also installs any profile-required AnchorV4 runtime and the final frozen Bridge facade whose V2 path queries the immutable inbox store. Legacy SignalService remains V1-only. The L1 side installs the same frozen facade, the non-proxy BridgeCreditRegistry, TerminalSignalVerifier, BridgeInboxAdapter and every official token vault's final capsule/reservation/claim facade before V2 send is enabled; every bridged-token admitted to V2 burn/mint mode has already installed and frozen `IRefundRestorableV2`. ProtocolVersionManager directly checks the deployed L1 components 1–3 and their constructor configuration, records those exact descriptor rows in the finalized manifest, computes the acyclic combined infrastructure/domain descriptor, seals the L1 adapter's domain once and burns that setter. The L2 release proof carries those same rows; its registrar directly checks local components 4–9 and recomputes the identical combined hash. Every legacy Bridge/vault proxy upgrade selector is no longer reachable and none of those facades delegatecalls another implementation. One non-proxy `InboxV2ActivationGate` is initialized false. InboxApplyRouterV2, the immutable-store batch and every L2 inbound-V2 process/retry/fail/expire path independently require its `active()` value; only AnchorV4's one-shot custom-system transaction may set it. This is an ordinary finalized L2 governance block; no L1 transaction is claimed to mutate an imported L2 state root. Its transaction bytes, receipts and resulting account/storage values are golden profile vectors.

Second, anyone advances the old Inbox until its finalized transition is quiescent; the gate requires the next proposal ID to equal the last finalized proposal ID plus one, the old core's finalized block-hash field, an empty native forced queue and no unsettled escrow. The caller supplies the canonical RLP of that final L2 header and fixed-depth MPT account/storage proofs under its state root for the exact AnchorV4, InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2, TerminalAccumulatorV2 and final Bridge-facade runtime code hashes; every EIP-1967 implementation/beacon slot and profile-defined fork-selector slot; the absence of any reachable upgrade/delegate target; InboxApplyRouterV2 permanent sender/registrar/namespace, `nextQueueIndex=0` and empty routes; inbox-store router/Bridge/registrar immutables, zero domain and empty mappings; empty release-authority/registrar records; accumulator empty domain writer,

count/root/completed-node store; each module's immutable gate address; and the gate's `active=false` state. The proof follows the exact proxy/router topology encoded in the profile and rejects an unlisted hop, delegate target or mutable upgrade path. Every terminal implementation and value must equal the immutable execution profile. The adapter requires its hash to equal that stored finalized hash, its `number<=UINT48_MAX`, and the legacy SignalService checkpoint at that exact number to equal its block hash and state root. This binds the height and state root that the current Inbox core does not store.

Third, the governance executor invokes the audited migration adapter. From that authenticated header and the frozen execution profile it constructs every initial field exactly:

```
l2BlockNumber       = header.number
tipHash             = H(RLP(header))
tipSlot             = header.timestamp - GENESIS_TIMESTAMP
stateRoot           = header.stateRoot
messageCursor       = 0
winningDataCommitment = H("slot-chain-migration-data-v2" ||
                          u256(settlementChainId) || u256(l2ChainId) ||
                          tipHash || stateRoot || terminalRoot ||
                          u64(terminalCount))
nextBaseFee         = profile.nextBaseFee(header)
nextExcessBlobGas   = profile.nextExcessBlobGas(header)
terminalRoot        = authenticatedAccumulator.root
terminalCount       = authenticatedAccumulator.count
canonicalizedAtBlock = block.number
```

It rejects a pre-genesis timestamp, arithmetic overflow, missing fork-required header fields or any profile mismatch. It freezes old proposal/forced ingress, imports this complete core into new PREACTIVE state, verifies the new queue's golden empty root/count/cursor and `dueAt(0)=UINT64_MAX`, and an empty BridgeInboxAdapter with no `QUEUED` record or deposit, invokes the router's sole `activateVersion` transition to authenticate the singleton queue and atomically register the new Settlement, writes the imported full core into that Settlement's sequence-zero history cell, changes PREACTIVE to ACTIVE, and stores the L1 activation commitment consumed by the first new AnchorV4 transaction. It does not write L2 code or storage. L1 kind-1 ingress is enabled only after the authenticated L2 code/authorization proofs and router-history authorization have all succeeded. That same atomic cutover enables kind-0 ingress, but both V2 send selectors and kind-1 ingress remain disabled. The old `sendMessage` selector and all L2-to-L1 sends retain exact V1 signals and events for every caller. The first post-cutover L2 block must begin with the custom AnchorV4 transaction. The circuit authenticates the L1 cutover commitment, exact imported parent and execution-profile hash. Its top-level sender is the reserved Anchor-system sender. AnchorV4 first calls `ProtocolReleaseAuthorityV2.activateRelease`; the authority verifies both `msg.sender=manifest.anchorV4` and `tx.origin=reservedAnchorSystemSender`. AnchorV4 then calls `TerminalDomainRegistrarV2.activateDomain`; the latter's atomic endpoint seal, inbox route and accumulator registration must succeed before that block's InboxApplyRouterV2 batch. The custom transaction type is invalid under the legacy fork and its reserved sender has no ordinary signing key, so neither a legacy block nor an EOA can preactivate the latch. Any first block that omits, duplicates, calls authority directly or through Bridge, substitutes an Anchor, or mismatches this transition is unprovable. All calls occur in one EVM transaction, so any failure rolls release and registration back together. Subsequent blocks

require the latch already true and cannot repeat activation. After that block becomes canonical and final, anyone submits its version/sequence and bounded `MptProofV1` registrar proof to BridgeDomainRegistry. Only that confirmation plus 214 L1 blocks enables `sendMessageV2`, `sendMessageFromVaultV2` and kind-1 ingress for the exact tuple. Thus no source credit can target an unregistered or merely proposed L2 endpoint.

Every later Settlement/profile migration uses a protocol-generated two-phase cutover. The delayed manifest arms a specific cutover generation and changes ACTIVE to MIGRATION_ARMED. This is one protocol-lifetime `MigrationGateV2`, not a caller-supplied quiescence Boolean. Settlement, registry, data-session, reward/preparation and both forced-ingress adapters all read the same `(generation,state)` word. Arming atomically closes new normal contexts/candidates, sessions, data posts, builder reservations and reward work; ingress returns SYNCED before retaining a deposit or queue write. Already-mature claim-only withdrawals remain callable.

The old Settlement finishes only the already-open boundary. An open normal window may commit its already-recorded best at its deadline; a newly due head cancels it. A current recovery proof may commit until that round's existing `expiresAt`; after expiry `sync()` cancels the round and is forbidden to increment its revision or open another episode. If neither is open, arm clears a merely armed normal context immediately. At and after that boundary, two authenticated cursors each refund/delete at most eight live data sessions and builder reservations per call, decrementing stored counters only with the corresponding ledger write. With at most 1,024 sessions and $64 \times 17 = 1,088$ reservations, at most 136 permissionless cleanup calls are needed after the existing boundary. READY is derived solely from zero counters, no normal boundary and no recovery; no calldata supplies those facts. The process preserves slash/evidence tranches in the protocol-lifetime registry, and leaves matured reward/capsule/refund claims callable on permanent claim-only surfaces. It then enters MIGRATION_READY before another `sync()` decision. A due queue head is not drained or discarded; old adapters receive SYNCED with their caller deposit refunded during READY and retry after activation.

One coordinator transaction then freezes the old immutable Settlement, static-reads its exact full Canonical, sequence, history-write block and MIGRATION_READY state, proves the exact same ForcedQueue address/root/count/ frontier/cursor/escrow/lastDueAt and InboxApplyRouterV2 cursor, imports all of it without narrowing into an empty PREACTIVE immutable Settlement, writes the imported version-namespaced history cell, registers its code/profile, and makes it ACTIVE. Any failure reverts the freeze and switch. The new Settlement's manifest must reproduce the protocol-lifetime kind-0 v2/kind-1 v10 decoder, global InboxApply row ABI and disposition transitions for every preserved queue entry; an incompatible decoder cannot activate. Existing kind-1 descriptors remain complete on L1, and an unavailable kind-0 preimage retains its original bounded expiry/discard path. The new Settlement's first history write requires a later L1 block, and its first `sync()` immediately opens recovery if the preserved head is due. Continuous builder submissions or a recovery backlog therefore cannot veto an armed upgrade. The operation contains only bounded non-reverting internal writes after its assertions; any revert rolls all contracts back. The adapter cannot cancel or convert legacy forced records; a nonempty legacy predicate simply blocks migration. Old bridge signals keep their original claim semantics, so the migration makes no unenforceable claim to enumerate all historical pending messages.

12 Parameters and enforced geometry

Parameter	Initial value	Enforced relation
L2 slot / schedule window	1 s / 384 slots	Aligned by genesis timestamp.
L2 height / timestamp	`uint48` checkpoint bound	Header number is consecutive and at most `UINT48_MAX`; header timestamp equals genesis plus signed slot.
`N_MAX`, `Q_MAX`	64 / 76	Six addresses fill 384 tickets.
`BOND_MAX`	$2^{192} - 1$	`bond*(Q_MAX+1)<2^256`.
Entry/exit/reservation ahead; moves	8 / 268 / 16 windows; 4 per window	Liability residence $<$ 268; capacity 4×268.
`D_SNAP`, `F_FINAL`	8 / 2 L1 epochs	Strictly exceeds scan + finality + seal margin + lookahead.
`H_LOOK`	768 L2 slots	Guaranteed by snapshot geometry with one-window slack.
`G_MAX`	3,600 L2 slots	Tier-1 parent gap; equals final-lag trigger and fits within the 12-window proof cap.
`W_SETTLE_SECONDS`	1,200 s	Timestamp deadline, never L1 block count.
`DELTA_TIP`	1,200 s	At least submit inclusion + skew = 144 s after the frozen escape slot.
`DELTA_FINAL_LAG`	3,600 s	At least activation + escape offset + submit + skew = 2,164 s.
`F_L1`; `T_DEPTH_MAX`	64 L1 blocks; 900 s	Stored recovery-anchor depth and assumed maximum time to reach it.
`ESCAPE_OFFSET`	1,900 s	At least depth-time bound + proof bound + margin.
`FORCE_DELAY`	1,500 s	At least settlement window plus `T_INCLUDE_MAX_SECONDS=120`.
Forced prefix	64 items; 1,048,576 B; 20m gas	Each item at most 131,072 B and 5m accounted gas; descriptors and one boundary remain durable.
Candidate caps	4,096 blocks; 12 windows; 1 anchor; 16 sessions; 2,100 records	Also 256 forced items, 4 MiB, 80m accounted gas.
Queue / range proof	depth 32 / at most 129 hashes	`queueCount<UINT32_MAX`; canonical contiguous range proof.
Same-L1 credit read	fixed-size record / bounded staticcall	Launch source equals settlement Ethereum; registry runtime, return length, record, source generation and Bridge liability are checked directly and atomically.

Bridge domain release	at most 64 new entries/version; append-only	Only atomic activation of a delayed immutable manifest registers entries; the historical registry has no global cap, delete, redirect or reinterpretation.
Source generation support	one immutable active-profile entry	The frozen endpoint tuple is usable only after `SUPPORT_FINALITY_BLOCKS`; no automatic implementation boundary exists.
Refund capsule	at most 256 words; ERC721 256 IDs; ERC1155 128 pairs	Persisted and hash-checked in the source vault before a V2 credit becomes NEW; claim work is bounded by the same exact profile cap.
Kind-0 validity / kind-1 enqueue	604,800 / 604,800 s	Bounds missing bytes before expiry or source cancellation.
`BRIDGE_PROCESS_TTL_SECONDS`	2,592,000 s	Starts at the immutable L2 pin; afterward anyone transitions NEW/RETRIABLE to FAILED without Message bytes.
Terminal accumulator/proof	depth 64 / 64 siblings / 2,048 B	Checked `uint64count<UINT64_MAX`; leaf commits index, domain, Bridge, credit and terminal; 65 bounded node writes per append and fewer than $2N + 65$ occupied node slots make current-state proof reconstruction explicit.
Settlement canonical history	256 version-sequence cells	$256 > \lceil (T_{include} + REORG\_MARGIN)/12 \rceil + 2$ with at most one canonical write per L1 block; the exact version/sequence tag prevents sparse L2 heights from choosing or churning a cell, and frozen versions never overwrite again.
Registration MPT proof	66 nodes/path; 600 B/node; 80,000 B; 8m gas	Canonical RLP and exact account/storage paths under the routed canonical state root; all bounds reject before an unbounded allocation or call.
Body chunks	9 / 1,142,748 B	Per-body chunks / maximum discretionary bytes.
`DATA_TTL`	86,400 s	Greater than candidate, settlement, proof and reorg path.
`REORG_MARGIN, EVIDENCE_DELAY`	1,800 / 86,400 s	Data resubmission and finite liability retention.
`SUPPORT_FINALITY_BLOCKS`	214 L1 blocks	Begins only after the domain's finalized canonical L2 registrar record is proven to L1; a frozen endpoint/domain cannot create credits earlier.
Live windows / sessions	268 / 1,024	Fixed rings with safe-eviction predicates and bounded GC.

EIP history / max arm age	8,191 / 255 blocks	$255 + \lceil (W_{settle} + CLOCK\_SKEW + 384 + EVIDENCE\_DELAY + REORG\_MARGIN)/12 \rceil + 2 < 8,191$; greater than every normal-context evidence and schedule path.

Launch tooling evaluates every inequality in both seconds and its native clock unit. Governance cannot set a value that violates a relation.

13 Production gates and verdict

The state-machine architecture is fixed enough for a prototype, but the repository is *not* an implementation-ready consensus specification. Its initial executable profile release bundle is absent. Before that status can change, one reviewed commit must contain all of the following normative artifacts and two independent implementations must reproduce every hash and execution result:

- canonical CBOR schema, exact profile bytes and `executionProfileHash`, plus a rejecting decoder corpus;
- an immutable manifest binding protocol version, chain IDs, activation, profile hash, verifier address/runtime hash and verifier-key hash;
- exact predeploy/proxy/facade topology, terminal runtime code hashes, constructor immutables and relevant storage selectors for AnchorV4, InboxV2ActivationGate, InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2, TerminalDomainRegistrarV2, TerminalAccumulatorV2, ActiveSettlementRouter, L1/L2 legacy SignalService, official refund vaults, frozen bridged-token restoration templates and both permanent Bridge facades, including proof that no upgrade selector or second delegate target is reachable, plus the exact chain placement and L1-direct/L2-direct/finalized-manifest validation trace for every infrastructure row;
- exact nonzero source/destination domain namespaces and vectors, append-only BridgeDomainRegistry authorization, BridgeCreditRegistry direct ABI/runtime/record layout, frozen source generations, Bridge aggregate and per-credit liability/status/pull-credit layouts, vault capsule/reservation/ claim layouts, per-Settlement sequence-tagged canonical-history rings and fixed-depth TerminalSignalVerifier vector grammar, permanent accumulator-node layout and append gas bound, byte-exact Bridge-kernel and destination-infrastructure descriptors, registrar/adapter/store one-shot bindings, finalized L2 registration proof, router activation ABI/invariants, direct-call limits and valid/invalid fixed-return corpus;
- exact unsigned system transaction type and reserved sender addresses; activation-latch and cutover proof; system nonces; separate direct/vault additive BridgeV2 selectors and bounded immutable inbox-store batch ABI, selectors and storage; fee handling, gas ceilings and all fork and header recurrence rules; and
- complete positive and negative vectors for parent, prestate, input, transactions, receipts, headers and poststate, as listed in §11, including domain/frozen-generation changes, unauthorized clones and foreign local-domain replay, enqueue/cancel and capacity races, PREACTIVE rejection, legacy preactivation attempts, first-block latch activation, direct-record absence, router-SYNCED refund/retry, old-domain routing, permanent-

pin independence from legacy upgrades, processing expiry, DONE release, FAILED recall, legacy SignalService pause, zero ordinary ETH/token quota, pooled-balance drain attempts, capsule claims, endpoint authorization, exact old-selector V1 compatibility, explicit V2 opt-in, terminal-root/count public-output binding, canonical-sequence collisions, historical proof after later appends, direct-sender/Bridge/foreign-Anchor release attempts, reserved-system-sender-to-Anchor activation, old-domain terminal append after new-domain activation, mixed old/new-domain global inbox batches and domain/Bridge proof substitutions, multi-credit batches and every V1/V2 replay path.

Those values are outputs of real compiled contracts, fork code and a real circuit/verifier key. Inventing placeholders in LaTeX or Python would make the profile hash look precise while leaving clients unable to construct the same escape block. This is missing normative consensus, not an empirical launch gate. Consequently a sound implementation-ready artifact cannot be completed by document-only revision; implementation and circuit work must now produce the bundle. Once it exists and is re-audited, production deployment remains blocked until every additional gate below produces a versioned artifact and passes:

1. **Proof performance.** Independent tier-1, tier-2 and worst-case tier-3 proofs, including 4,096 blocks and 2,100 data records, finish within 900 seconds on the published minimum hardware. Peak RAM, recursion setup and failure recovery are measured; a miss requires changing parameters and another design review.
2. **Contract gas.** `sealWindow`, `postData`, normal submit, `sync`, direct source-credit enqueue, recovery submit, slash and garbage collection fit current L1 gas limits with 30% margin. Fuzzing includes empty registry, zero seat, full data session, full queue prefix and maximum history age.
3. **Cryptographic conformance.** Solidity, Go, Rust and circuit code match the Keccak/EIP-712 golden vectors, direct same-L1 Bridge registry ABI and runtime-hash checks, KZG Fiat–Shamir transcript and exact body manifest. Two independent implementations generate every vector.
4. **State-machine verification.** Stateful fuzzing models EVM rollback, same-block ordering, reorg replay, stale versions, activation/close coincidence, message consume-and-discard and storage reclamation. The executable models in Appendix D remain green.
5. **Economics.** Auction bond, per-window tranche, forced-envelope deposit, storage bond and proof credits are calibrated under blob/L1 fee spikes. The published liveness claim explicitly becomes conditional above fee caps.
6. **Operations.** At least two independent prover stacks and three independently administered canonical archives, plus public data-session mirrors, schedule sealers and recovery bots, complete a shadow-fork outage exercise. One archive serves a clean node, after deleting its state, body, receipt *and code* databases, from only the latest finalized package while the other two are removed. The exercise also removes every builder and seat, forces a user transaction, reorgs the first recovery attempt, and still reaches finality. Loss and restore drills verify that unavailable prestate is never misreported as bounded recovery. For every supported source domain, an analogous drill creates an old immutable credit record, removes every message-archive copy, exercises source cancellation, then recreates a message, orphans its direct enqueue transaction and replays it from the surviving record. It also exercises two separately registered frozen source generations, attempts and fails to rewrite the Bridge/vault implementation slots, pauses legacy SignalService and zeros ordinary quotas,

replays a pin through a second funded Bridge, expires an unprocessed pin and proves that a superseding profile cannot remove an old domain or endpoint entry.

7. **External review.** Separate proof-system, Solidity, bridge and economic audits close all critical/high findings; the migration rehearsal and emergency governance path are in scope.

Verdict. *The earlier fallback architecture was not implementation-ready: it depended on a scheduled builder, could revert activation inside a rejected submission, made payment a commit precondition, and could not bind whole-window data with a six-blob cap. This revision removes those dependencies, fixes a single consensus path, binds bridge credits to their source application, and defines bytecode-complete recovery packages. Its bridge integration now freezes every custody/recovery facade, keeps legacy callback applications on V1, binds the local destination, and makes V2 terminal proof/refund paths transitively pause- and quota-independent. No additional critical/high architecture defect is known under the stated L1/source-chain, proof, funding, canonical-prestate and source-state-witness availability assumptions. Nevertheless the design is* not implementation-ready*: the initial executable profile bundle above does not exist. This audit therefore stops at the document-only impossibility boundary, instead of falsely promoting test fixture hashes into production consensus.*

A Normative commitment encodings

All integers in packed commitments are unsigned big-endian at the named width; addresses are 20 bytes, hashes are 32 bytes, and ASCII domains have no length prefix or trailing NUL. `H` is Ethereum legacy Keccak-256. No `abi.encodePacked` call may contain a dynamic field. Raw transaction bytes are represented by `H(rawTx)` plus an explicit `u32(byteLength)`.

The canonical core/base, candidate, schedule list, sealed-session list and execution outputs use these exact encodings:

```
coreHash = H("slot-chain-core-v3" || u64(l2BlockNumber) || tipHash ||
  u64(tipSlot) || stateRoot || u64(messageCursor) || winningDataCommitment ||
  u256(nextBaseFee) || u64(nextExcessBlobGas) || terminalRoot ||
  u64(terminalCount))
baseCanonicalHash = H("slot-chain-canonical-v2" || coreHash ||
  u64(canonicalizedAtBlock))
H("slot-chain-candidate-v2" || baseCanonicalHash || u16(n) ||
  (u64(slot) || blockStructHash || blockHash || bodyRoot ||
   dataManifestRoot || u64(messageEnd))*)
H("slot-chain-schedule-list-v1" || u8(count) ||
  (u64(window) || entryRoot || seed)*)
H("slot-chain-session-list-v1" || u8(count) ||
  (sessionId || u16(recordCount) || root)*)
winningDataCommitment = H("slot-chain-winning-data-v1" ||
  candidateCommitment || sessionRefsCommitment)
H("slot-chain-outputs-v2" || stateRoot || transactionsRoot || receiptsRoot ||
  logsBloomHash || withdrawalsRoot || terminalRoot || u64(terminalCount))
```

Here $n \leq 4096$; schedule windows are ascending and sessions are ascending by ID. `blockStructHash` is the EIP-712 struct hash of the exact signed `SlotChainBlock` fields in §5.1; tier-3 recovery uses the same struct fields without a signature. These hashes are respectively the statement's `baseCanonicalHash`, `candidateCommitment`, `scheduleRootsCommitment`, `sessionRefsCommitment` and `executionOutputsCommitment`. Settlement must recompute the displayed `winningDataCommitment` before verifier dispatch and again before the canonical write; unsorted or duplicate lists reject.

The 64-cell registry leaf at index i is

```
H("slot-chain-registry-leaf-v1" || u8(i) || u8(present) ||
  address20 || u192(bond) || u64(registrationIndex) ||
  u64(effectiveL2Slot) || bytes32(trancheRoot) || u64(tombstonedAtL2Slot))
```

An empty cell has `present=0` and every following byte zero. A present cell has `present=1`; `UINT64_MAX` means not tombstoned. At tree height $h = 0, \ldots, 5$, the parent is:

```
H("slot-chain-registry-node-v1" || u8(h) || left || right)
```

There is no padding because there are exactly 64 leaves.

Admission position $i < 1,136$ is:

```
H("slot-chain-admission-leaf-v1" || u16(i) || u8(present) ||
```

```
  u8(location) || address20 || u192(bond) || u64(registrationIndex) ||
  u64(effectiveL2Slot) || u64(tombstonedAtL2Slot))
```

Location is 1 for active and 2 for liability; an empty leaf has present and all following fields zero. Positions $1,136 \ldots 2,047$ and unused positions are canonical empty leaves. Its eleven levels use `H("slot-chain-admission-node-v1"||u8(height)||left||right)`. `admissionVersion` increments before publishing the corresponding root; the pair is stored together and never reconstructed from a newer root. Tranche roots occur only in the active registry and liability storage records, not in this stable signer-identity commitment.

The omission-free ranked-entry commitment for window W has exactly 64 leaves:

```
H("slot-chain-entry-leaf-v1" || u8(rank) || u8(present) || address20 ||
  u192(bond) || u64(registrationIndex) || u64(effectiveL2Slot) ||
  u64(tombstonedAtL2Slot) || bytes32(trancheLeafHash))
```

followed by six levels of `H("slot-chain-entry-node-v1"||u8(height)||left||right)`. Empty ranks have all fields zero. The schedule circuit derives all 384 tickets from this root and seed; a schedule list cannot substitute a different set.

Each registry cell's tranche ring is a 512-leaf tree. Leaf index j is:

```
H("slot-chain-tranche-leaf-v1" || u16(j) || u64(window) || u8(state) ||
  u192(amount) || u64(liableUntil))
```

An empty leaf has `window=UINT64_MAX`, state `EMPTY=0` and remaining fields zero. Other states are `FREE=1`, `RESERVED=2`, `LIABLE=3`, `RELEASED=4` and `SLASHED=5`. A ring-index collision may be overwritten only from RELEASED or SLASHED after its absolute release predicate; the write installs the new exact window and FREE/RESERVED state atomically. `liableUntil=0` for EMPTY/FREE; reservation sets it exactly to `evidenceDeadline(window)+REORG_MARGIN_SECONDS`, and LIABILITY preserves that value. RELEASED/SLASHED preserve it until safe overwrite. Its nine node levels use `H("slot-chain-tranche-node-v1"||u8(height)||left||right)`. The snapshot supplies one inclusion proof for index `Wmod512` for every active cell, including canonical empty and `FREE` leaves.

Kind-0 forced leaf i is legacy Keccak over this exact packed sequence:

```
"slot-chain-force-user-v2" || u32(i) ||
address20(sender) || u64(nonce) || u256(l2ChainId) || bytes32(rawTxHash) ||
u32(byteLength) || u64(gasLimit) || u64(accountedGas) ||
u256(maxFee) || u64(validUntil) || address20(refundAddress) ||
u64(enqueuedAt) || u64(dueAt) || u256(deposit)
```

A kind-1 leaf is:

```
"slot-chain-force-bridge-v10" || u32(i) || msgHash ||
u256(srcChainId) || sourceDomainId || u64(srcEpoch) || address20(srcBridge) ||
bridgeExecutionHash || u64(emittedAtBlock) || destinationDomainId ||
u256(destChainId) || u64(enqueueBy) ||
address20(sender) || address20(srcOwner) || address20(destOwner) ||
u256(value) || u64(fee) || calldataHash || u8(refundMode) ||
address20(refundVault) ||
```

```
refundCapsuleHash || escrowId ||
u32(byteLength) || u64(accountedGas) || address20(refundAddress) ||
u64(enqueuedAt) || u64(dueAt) || u256(deposit)
```

Its application has no expiry; no `validUntil` field exists.

For `forcedDescriptorCommitment`, `descriptorBytes` is exactly the corresponding leaf sequence beginning immediately after `u32(i)`; the list already supplies index and kind, so neither leaf ASCII domain nor index is duplicated inside it. Its fixed byte length is 220 for kind 0 and 533 for kind 1. The list encoding and optional first excluded boundary are defined in §8. The circuit prepends the correct leaf domain and index to reconstruct every Merkle leaf; any other length rejects.

The acyclic `canonicalBridgeKernelDescriptor` is exactly 116 bytes:

```
u16(kernelVersion=1) || u8(DIRECT=1) || u8(CAPSULE=2) ||
u64(MAX_BRIDGE_ENQUEUE_DELAY=604800) ||
u64(BRIDGE_PROCESS_TTL_SECONDS=2592000) ||
bytes32(bridgeV2AbiHash) || bytes32(statusTransitionHash) ||
bytes32(custodyRulesHash)
```

The three final hashes commit byte-exact selector/tuple/event grammar, status transition table and liability/reserve/pause rules in the release bundle. None may contain a source/destination domain, infrastructure hash or full `executionProfileHash`. The kernel hash is

```
bridgeKernelProfileHash =
  H("slot-chain-bridge-kernel-profile-v1" || u32(116) ||
    canonicalBridgeKernelDescriptor)
```

`canonicalFrozenBridgeDescriptor` then has this exact 156-byte packed grammar:

```
address20(bridge) || address20(creditRegistry) ||
address20(terminalVerifier) ||
bytes32(facadeRuntimeHash) || bytes32(storageLayoutHash) ||
bytes32(bridgeKernelProfileHash)
```

Every address and hash is nonzero and `bridge=srcBridge`. The registry stores the whole manifest-authorized descriptor and computes:

```
bridgeExecutionHash = H("slot-chain-frozen-bridge-execution-v2" ||
  u32(156) || canonicalFrozenBridgeDescriptor)
```

The credit registry exposes this fixed descriptor for the source generation; there is no block-indexed executor. Direct admission checks the frozen runtime and record on the same L1. No source state root, beacon walk or caller-chosen topology proof exists in that path.

`canonicalDestinationBridgeDescriptor` has this exact 196-byte packed grammar:

```
address20(bridge) || bytes32(facadeRuntimeHash) ||
bytes32(storageLayoutHash) || bytes32(bridgeKernelProfileHash) ||
address20(inboxCreditStoreV2) || address20(terminalAccumulatorV2) ||
address20(activationGate) || address20(terminalDomainRegistrarV2)
```

Every field is nonzero and `bridge=destinationBridge`. It commits the destination facade separately from the infrastructure record so its custody, terminal handshake and storage layout cannot be inferred from a source-only descriptor. The hash is:

```
destinationBridgeExecutionHash =
  H("slot-chain-destination-bridge-execution-v1" || u32(196) ||
    canonicalDestinationBridgeDescriptor)
```

`canonicalDestinationInfrastructureDescriptor` has exactly nine component records in this order:

```
BridgeInboxAdapter, ActiveSettlementRouter, TerminalSignalVerifier,
InboxApplyRouterV2, InboxCreditStoreV2, ProtocolReleaseAuthorityV2,
TerminalDomainRegistrarV2, TerminalAccumulatorV2, destination Bridge
```

Each record is

```
address20(component) || bytes32(runtimeHash) || bytes32(configHash)
```

so the descriptor is exactly 756 bytes. For kind $k = 1, \ldots, 9$ in that order,

```
configHash = H("slot-chain-component-config-v1" || u8(k) ||
               u16(byteLength) || configBytes)
```

Every component implements `componentConfigHashV2()viewreturns(bytes32);` a checking side requires exactly 32 bytes equal to this value under the manifest-pinned runtime hash. Empty, short, trailing or malformed returndata rejects. L1 checks rows 1–3 directly and L2 checks rows 4–9 directly; neither substitutes a Boolean result for the fixed call and hash comparison. `configBytes` is exactly:

1. `address20(bridgeCreditRegistry) || address20(sourceBridge) || address20(activeSettlementRouter) || address20(protocolVersionManager)`
2. `address20(protocolVersionManager) || address20(forcedQueue) || bytes32(routerNamespace)`
3. `address20(activeSettlementRouter) || u8(64)`
4. `address20(terminalDomainRegistrarV2) || address20(inboxSystemSender) || bytes32(routerNamespace) || u8(64)`
5. `address20(inboxApplyRouterV2) || address20(destinationBridge) || address20(activationGate) || address20(terminalDomainRegistrarV2)`
6. `address20(anchorSystemSender) || bytes32(manifestNamespace)`
7. `address20(protocolReleaseAuthorityV2) || address20(inboxApplyRouterV2) || address20(terminalAccumulatorV2)`
8. `address20(terminalDomainRegistrarV2) || u8(64)`
9. `address20(inboxCreditStoreV2) || address20(terminalAccumulatorV2) || address20(activationGate) || address20(terminalDomainRegistrarV2) || bytes32(storageLayoutHash)`

The runtime semantics for kinds 1, 5, 7 and 9 fix the stated one-shot initializer authority, zero prestate and authority-bit burn. Kind 4 contains only protocol-lifetime constants and never

an endpoint Store, Bridge or gate; endpoint-specific targets enter only through the registrar's append-only route. `configBytes` excludes the derived `destinationDomainId` itself to avoid self-reference. Every component address/runtime/config hash is nonzero and equals the separately named domain/profile field. The infrastructure commitment is:

```
destinationInfrastructureHash =
  H("slot-chain-destination-infrastructure-v2" || u32(756) ||
    canonicalDestinationInfrastructureDescriptor)
```

Rows 1–3 name L1 contracts and rows 4–9 name L2 contracts. At manifest activation, L1 ProtocolVersionManager directly checks rows 1–3 against L1 code/configuration and commits all nine rows. The finalized manifest proof authenticates those L1 rows to ProtocolReleaseAuthorityV2; TerminalDomainRegistrarV2 directly checks only rows 4–9 against L2 code/configuration. Both sides recompute the same 756-byte descriptor and hash. No contract is required or permitted to treat a cross-chain address as locally inspectable code.

The depth-32 vector empty leaf is `H("slot-chain-force-empty-v2")`. A parent at height $h = 0,\ldots,31$ is `H("slot-chain-force-node-v2"||u8(h)||left||right)`. The queue commitment is `H("slot-chain-force-root-v2"||u64(count)||treeRoot)`; leaves at indices `count..2^32-1` are empty. The canonical range proof recursively visits the smallest tree covering `[start,end]` (including a boundary leaf when `end<count`); it emits in DFS order exactly one hash for each maximal outside subtree. The verifier rejects extra/missing nodes, a nonempty padding leaf, or a reconstructed root/count mismatch. Append at old index i starts with the new leaf at height zero, repeatedly hashes `frontier[h]` on the left while bit h of i is one, stores the carry in the first zero-bit frontier position, increments count, and reconstructs all higher right subtrees from `FORCE_EMPTY[h]`. This is the same root as the full vector and uses exactly 32 frontier words.

The terminal vector uses depth 64. Its empty leaf is `H("slot-chain-terminal-empty-v1");` a parent at height $h = 0,\ldots,63$ is

```
H("slot-chain-terminal-node-v1" || u8(h) || left || right)
```

and leaf index i is

```
H("slot-chain-terminal-leaf-v1" || u64(i) || destinationDomainId ||
  address20(destBridge) || creditId || u8(terminal))
```

with `terminal=1` for DONE or 2 for FAILED. The committed root is

```
H("slot-chain-terminal-root-v1" || u64(count) || treeRoot)
```

The accumulator stores `completed[height][nodeIndex]` only when that entire aligned subtree first becomes full; height zero entries are immutable leaves. Missing entries are never interpreted as empty for an occupied complete interval. To append old count i, it writes the leaf, then for each trailing one bit of i combines the already completed left sibling with the carry and stores the newly completed parent. It stores no mutable partial parent. The current root is folded from the frontier of complete left subtrees and the precomputed `TERMINAL_EMPTY[height]` values.

`getNodeAt(count,height,index)` is defined for the aligned interval $[index\,2^{height}, (index+1)2^{height})$: it returns the canonical empty node when the interval starts at or after `count`, the

immutable completed node when it ends at or before `count`, and otherwise recursively hashes the two children across the unique partial boundary. `getProofAt(count,i)` uses that function for exactly 64 siblings in ascending height. It therefore reconstructs any historical prefix named by a retained Settlement cell even after later appends; current mutable nodes cannot corrupt an old proof. An append performs one leaf write plus at most 64 carry writes and 65 hashes, fewer than two completed-parent writes amortized. The union of occupied slots after N appends is below $2N + 65$. A verifier rejects an index not below the committed count, any missing occupied completed node, wrong leaf field, wrong proof length or root/count mismatch.

A data-chunk leaf is:

```
H("slot-chain-data-leaf-v1" || bytes32(sessionId) || u16(index) ||
  bytes32(versionedHash) || bytes32(fullBodyRoot) || u16(blockOrdinal) ||
  u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
  bytes32(chunkRoot) || address20(publisher) ||
  u64(validUntil) || u256(z) || u256(y))
```

MMR append merges equal-height peaks as `H("slot-chain-data-node-v1"||u8(height)||left||right)`. Peaks are stored left-to-right in strictly decreasing height. The root bags them rightmost-to-leftmost as `H("slot-chain-data-bag-v1"||u16(count)||u8(peakCount)||(u8(height)||peak)*)`. An inclusion proof provides the leaf index, merge siblings bottom-up with a left/right bit at each height, then every other peak in this bag order; duplicate heights, a count inconsistent with peak heights, or unused proof elements reject. The empty MMR is the bag hash with zero count and zero peaks.

A manifest leaf is:

```
H("slot-chain-manifest-leaf-v1" || u16(position) ||
  u16(blockOrdinal) || sessionId || u16(recordIndex) ||
  u16(chunkIndex) || u16(chunkCount) || u32(chunkByteLength) ||
  fullBodyRoot || chunkRoot)
```

Its next-power-of-two tree uses

```
H("slot-chain-manifest-node-v1" || u8(height) || left || right)
H("slot-chain-manifest-root-v1" || u16(count) || treeRoot)
```

for the node and final root respectively; padding uses the domain-separated empty leaf. For zero entries, `treeRoot=H("slot-chain-manifest-empty-v1")`. The zero-discretionary- transaction body is the ordinary body encoding `u32(0)` and therefore `H("slot-chain-body-v1"||u32(4)||u32(0))`. Tier 3 requires exactly those empty body/manifest values and the zero-count session-list commitment. This tree is instantiated separately for every block. `position` starts at zero inside that block; `blockOrdinal` remains the candidate-wide block index. A two-block candidate therefore commits two signed manifest roots and never hashes their leaves into one shared tree. The InboxApply disposition commitment is:

```
H("slot-chain-dispositions-v1" || u64(start) || u64(end) ||
  u16(count) ||
  (u64(queueIndex)||u8(disposition)||u32(txIndex)||resultHash)*)
```

Here `UINT32_MAX` means no Ethereum transaction.

The recovery ID is the encoding in §7. Schedule leaf/root, seed and tie hashes are as in §4; body and blob encodings are as in §6. `commitment-model.py` is the normative executable fixture. Its initial vectors are:

```
TYPEHASH       ee6a8c8e31e8245cd527869508f6e464d6084893991203876f734d1855aed87c
registryRoot   0bf297d7b9b6a5529a319a06cb08484923a89bab15d51f8baeaf5c30bebdf3fd
admissionRoot  3bf2dcaf78292c832108e29205bf99cc2d22137a0545e4528d8da7309d4b482b
entryRoot      acee83a690b868a4a7960c55a9f7228f91cad26b704e24106d4db87e9c7a8f34
forcedRoot     ab03f105ee7d619fb2c81d31b38760720dff2cb35471bc28fdc063c31f71bd67
sourceDomain   9f207cb5f32747635be6496e957827270c0b62aa5a7d0bd39657e28c0fedf7b1
destDomain     78cd6a4469c1e6ba8e7e3719955e330e68a66c48b70d9fc02600fa9244bb0eba
bridgeKernel   371700b6908037409059b484e1eff1a1511464447af6c67ea98cc96d35443cc6
bridgeExec     1d3ceea2019b62ab462d82680915fe06633fa0f171caf5ddc55d706e458b0ba2
destBrExec     66c6a4a377d3719f2ca03e47fdb2e6bc5a2744ad2bc79baf5e41fbcc67c1d72d
destInfra      d0933ad1f03a0ebe485c8372d9d4efa6d8ec3a1655353830bf27d5ba82a857ca
relManifest    c471ab85901ae32270ad08f89ce49285ca8077b373d9f1adc66c5a42cd4f06d6
destReg        74c48390918a5cb1c131f5a821f035aee1af7fbd37259606282701b3e52c9693
regBase        20b3dfc457e3cecf32b0c047177351f0814e426c1548e87b79f58830655810c3
regSlot        dfa6283b763bbadeb604401a78e2fefeddb72000addcdb94ed2e3de5cc69846b
regTrie        200031adff46d90b1cd5c67ff8e31098235d1dddb08ec98b0d20f5f8660c0ac8
relBase        a0b7a29a75032f37561036cd3741e7b375213309367f37b5ffec4ad55cf6154f
relSlot        719bb73ba856aeab1b203e322bfefc6d84a4c41a3222bcf1634b1b44e5b9aba8
relTrie        dac8109059d03da2ad16ac3acc50d2e58897b8c3a7f6889ae77bfb20737e87a2
routeConfig    faa8da419cafcc9ef8eb8a4d1f6c49c1626a72d8e8cab4826ba7cc887c2df526
bridgeLeaf     6d0fc74112a9208b01c1cf652d2aae15fda421e71efc7d5f0333b8028908366d
creditId       80f6872de8b64760514c16f68f4dd62828bf2f715cf7ceaecb4e28db077a0c85
escrowId       dda628a9c5230f1493589515a189c645e422f5ef104ac83af5200c3a05da1617
inboxSlot      bd52439306c8d3a956854adb849c42bb9c2b3810e736c6a16b65b810ce876452
terminalDone   308c81b638c0d7027182f6e75dde3fa7b982b2b33f37ae5e7e25f0337cc51bb4
terminalFail   405fc99d08d18751652bf01196c948d5c713e65d9154a8fc9af592b11b14fd81
terminalRoot2  430278faec9d7c7610c7a659b12bdc7bf3c8f16c5f6e01030e86900b59d5cdae
emptyTermRoot  0a0a8606a440497456ab8cac6894ff589cd7a6464809113ea62b7affe1429cc2
bridgeResult   ca4e2ae3710db4146239a2ca871eff81822b03a493ecb984872bd4e097ae98d2
manifestRoot   417be737a57e38eb410f2d6e65c77ee19d5c314cdaf432067861c6a36c6a990f
normalContext  bdec611a765250c0e964b4e3c95c648fec0c126a0f2c8f8fc3c13f691ede455b
migrationData  a3588b50c1f4e768cb2c35a622452527ab3cb953520ef018af3ff2d2362b86a5
winningData    9b7418cffd7b88f9f3d3799fb287cd47dee2e34edb1d41590dbdd12f0859a34d
statementHash  17c45efbdb95e3511eb9e96d054fb3d1a4b4dec3b7aee5c499741b44863de0b6
recoveryId     b710e3a0629fa9ba438e5661ece58060cf547a0444d8b59b266fd500a4020c3a
bodyRoot       0f4e161a46c8b18c2a86f23a0a4e7169a838a12af8b389f65e97b547a99707e9
```

Release CI must reproduce every vector in Solidity, Go, Rust and the circuit.

B Normative transition ordering

Every external candidate entry point implements the following structure:

```
function armNormalContext() returns (Status) {
```

```
    if (mode == PREACTIVE) return REJECTED_PREACTIVE;
    if (sync()) return SYNCED;
    if (mode == NORMAL_ARMED && block.number > armBlockNumber + 255) {
        armBlockNumber = block.number;
        emit NormalContextRearmed(armBlockNumber);
        return REARMED;
    }
    if (mode != NORMAL_IDLE) return IGNORED;
    armBlockNumber = block.number; // no caller-selected hash or anchor
    mode = NORMAL_ARMED;
    return ARMED;
}

function activateNormalContext(armHeaderRlp) returns (Status) {
    if (sync()) return SYNCED;
    if (mode != NORMAL_ARMED) return IGNORED;
    age = block.number - armBlockNumber;
    require(age > 0 && age <= 255);
    armBlockHash = blockhash(armBlockNumber);
    require(keccak256(armHeaderRlp) == armBlockHash);
    freeze base, stable admission pair and parsed arm header;
    freeze minimum slot, deadline and landing horizon;
    mode = NORMAL_OPEN;
    return ACTIVATED;
}

function submit(input) returns (Status) {
    if (mode == PREACTIVE) return REJECTED_PREACTIVE;
    if (sync()) return SYNCED;     // successful transaction, never revert
    if (mode == NORMAL_OPEN) return submitNormal(input);
    if (mode != RECOVERY) return REJECTED_NO_CONTEXT;
    return submitRecovery(input);
}

function sync() returns (bool changed) {
    if (migrationGate.state == MIGRATION_ARMED) {
        if (mode == RECOVERY && now > round.expiresAt) {
            cancelCurrentRecoveryWithoutRolling();
            changed = true;
        } else if (isNormalMode(mode) && forcedHeadDue &&
                   normal boundary open) {
            cancel normal window;
            changed = true;
        } else if (normal deadline matured) {
            settleOrCancelAlreadyRecordedBest();
            changed = true;
        }
        refundAtMost8DataSessions();
```

```
        refundAtMost8BuilderReservations();
        if (authenticated counters are zero && no boundary/recovery)
            migrationGate.state = MIGRATION_READY;
        return changed; // no context opens or rolls while armed
    }
    if (migrationGate.state == MIGRATION_READY) return false;
    if (mode == RECOVERY && now > round.expiresAt) {
        rollRound();               // same episode/base/burn; new target
        return true;
    }
    if (mode in {NORMAL_ARMED,NORMAL_OPEN} && forcedHeadDue &&
        (no deadline || normal window immature)) {
        cancel normal window;
    } else if (normal deadline matured) {
        if (best exists) {
            liveBoundaryDueAt = ForcedQueue.dueAt(best.endCursor);
            dataLive = now + REORG_MARGIN_SECONDS <= best.minDataExpiry;
        }
        if (best exists && liveBoundaryDueAt > now && dataLive)
            commit(normalBest);
        else cancel normal window;
    }
    if (isNormalMode(mode) &&
        (finalLagBreach || forcedHeadDue)) {
        openRecoveryEpisodeAndRound();
        if (finalLagBreach) terminate incumbent once;
        return true;
    }
    return changed;
}
```

No transfer, reward calculation, unbounded loop or caller-controlled external call occurs in canonical settlement. Optional reward materialization is a later transaction in a separate module.

C Rejected alternatives

1. **Heaviest fallback.** Rejected because an observer cannot prove it has seen the globally heaviest off-chain tail; transparent races can prevent a stable target. Recovery is episode-bound first-valid.
2. **Scheduled-builder-only fallback.** Rejected because the same cartel that stalls normal production also controls recovery. Tier 3 is deterministic and unsigned.
3. **Promote the pre-activation normal best.** Rejected because it was accepted under different cutoff/version semantics. Only a mature best that covers the required forced prefix closes; immature state is canceled.
4. **Atomic reward-funded commit.** Rejected because an empty seat or vault becomes a consensus deadlock. Reward materialization is a separate non-authoritative transaction.

5. **One same-transaction blob set.** Rejected because Ethereum exposes at most a handful of blobs per transaction while a candidate spans thousands of blocks. Publisher-owned authenticated sessions separate posting from proof.
6. **One KZG opening at a prover-chosen point.** Rejected because it does not soundly tie declared bodies to the blob. The challenge commits to both blob hash and body root, and the circuit recomputes the evaluation and body root.
7. **Arithmetic snapshot slot without EIP-4788 parent semantics.** Rejected. A bounded forward carrier search authenticates the parent source or fails closed.
8. **Bond as insurance.** Rejected without an on-chain reservation ledger. The tranche is deterrence only.
9. **Block-indexed delegatecall Bridge executors.** Rejected because an executor shares the permanent proxy's storage context and can overwrite the EIP-1967 implementation, liability, status or capsule slots. Custody/recovery facades are frozen; new semantics require a new endpoint and domain.
10. **Legacy SignalService as the V2 terminal verifier.** Rejected because its verification entry points are guardian-pausable and versioned. The V2 pin store is separate and immutable; the terminal verifier proves a stable application-level vector leaf against a proved canonical root.
11. **Implicit V2 behind the legacy selector.** Rejected because it silently changes events, signals, relayer/status APIs and EOA behavior, while a vault-wide interface probe cannot choose V1 per asset. `sendMessage` is always exact V1; direct and capsule V2 use distinct additive selectors.
12. **External checkpoint copier or L2-height-indexed publication ring.** Rejected because a caller can front-run or omit the copy, while a candidate may advance 4,096 L2 blocks and collide modulo 256 in one L1 publication. Each immutable Settlement writes its full canonical tuple internally under a checked version/sequence; no external publication call exists.
13. **Frozen EVM account/storage terminal proof.** Rejected because a future state-proof format or verifier defect can strand already-queued credits. Canonical settlement instead carries a profile-independent depth-64 terminal vector root/count, while governance remains only the pre-existing validity- verifier safety root.

D Executable consensus models

`lookahead-model.py` reports 36 assertions over legacy Keccak, EIP-4788 carrier/parent selection, immutable seal deadlines, post-seal tombstones, partial/empty registry sealing, eligibility-before-ranking, capped D'Hondt and feasible placement, including absolute clock conversion, execution-block finality depth, fixed seed and complete-schedule vectors.

`settlement-window-model.py` reports 166 assertions over PREACTIVE/migration, early-return semantics, force-due precedence, renewable recovery, no-seat escape, exact slot/header time, durable-descriptor prefix predicates, immutable anchor chronology, O(1) due boundaries, late close, sealed data sessions, frozen admission pairs, bounded replacement/liability generations, tranche release units, atomic direct bridge enqueue and capacity races, persistent SYNCED refund/retry, enqueue/cancel ordering, source-generation/domain support, immutable destination pins, processing expiry, aggregate ETH/token reserves, quota-independent refunds, immutable authorization versus mutable liability, frozen Bridge/vault

facades, exact legacy-selector V1 compatibility and selector-explicit direct/vault V2 opt-in, immutable pin storage, finalized registrar authorization, authenticated router switching, local destination binding, refund capsules, sequence-retained accumulator proofs and bridged-token restoration, pause-independent escape, L2 activation gating, replay and timing geometry. Cryptographic booleans in that model are not cryptographic evidence.

`commitment-model.py` reports 150 vectors/properties for exact EIP-712 domain/struct/digest, canonical and statement hashes, registry/admission/entry/ tranche commitments, forced descriptors, vector/range proofs, session MMR and per-block manifests. It also covers source/destination domains, acyclic Bridge-kernel/frozen-facade and nine-component infrastructure descriptors, permanent completed-subtree terminal nodes, historical-prefix proofs and complete credit leaves, credit/escrow/inbox IDs, terminal leaves/vector roots and atomic ordered batches, dispositions, recovery ID, Fiat–Shamir input, body chunks and blob codec. The valid KZG-opening, signature-recovery, independent-implementation and proof gates remain mandatory.

E Security invariants checklist

1. Canonical state changes only after one valid proof and only from its stored base tuple.
2. A sync transition cannot be rolled back by validation of the same call's candidate.
3. No payment, seat or standby is required for canonical commit.
4. Recovery candidates bind one live episode/revision, the current base tuple, one stored prior-block anchor at required depth and one frozen queue target. Expired rounds are permissionlessly renewable without another burn.
5. A due forced head advances in the next canonical block; appends omitted by a frozen recovery round cannot become due until that round expires. Prefix count, bytes and accounted gas are independently bounded.
6. Tier 3 has no builder signature, discretionary transaction, arbitrary coinbase or blob body.
7. Every block header timestamp equals genesis plus its signed slot. Every tier-1/2 block has a valid scheduled signature under the frozen admission version/root pair, strict slot progress and the candidate's one authenticated anchor no later than its L2 time.
8. Every discretionary body byte is covered exactly once by a unique, unexpired authenticated data record.
9. Schedule inputs share one authenticated execution payload and use exact Keccak encodings.
10. A tombstoned generation cannot re-enter while retained; safe later address reuse receives a fresh registration index only after every evidence deadline. Each window has an independent slashable tranche and finite release state.
11. Candidate, window, data-session, Merkle range proof, queue count/bytes/gas, history and storage-retention work is bounded by consensus constants and safe eviction.
12. L1 reorg replay removes canonical state, penalties and credits from orphaned transactions together.
13. An armed later migration shares one generation gate across ingress, Settlement and transient ledgers, opens no new context, ends the current boundary without recovery renewal and reaches READY after at most 136 bounded cleanup calls. Cutover freezes the old Settlement, imports canonical/bridge cursors into the inactive target and switches the one active queue router through its sole manifest-authenticated transition. At launch, V2 L2 entry points remain disabled under legacy rules; the first custom post-cutover Anchor transaction atomically registers/seals the destination and activates their shared gate. Source V2 waits for a finalized proof of that record plus 214 L1 blocks.
14. A kind-1 credit is enqueued only by direct same-L1 comparison with an append-only CreditRecord, frozen source generation, source-approved destination domain and live liability at the permanent

source Bridge facade. Enqueue and queue deposit are atomic; a reorg removes both observation and append. Direct and capsule refund modes have distinct selectors and committed mode/vault fields. Missing Message bytes cause bounded source cancellation before enqueue or destination expiry after the immutable pin. The exact destination domain, Bridge and terminal accumulator leaf bind recall; the accumulator's current node store reconstructs every old proof; every destination action checks its local domain and `address(this)`, and no L2 nonempty source-proof path exists. Terminal proof and reserved refund paths have no transitive guardian-pause or ordinary-quota dependency.